

THE M-TENSOR FORMAT: OPTIMALITY IN NONLINEAR HIGH DIMENSIONAL REGRESSION WITH SCARCE DATA *

RÉMI CLOAREC[†], SEBASTIAN RODRIGUEZ[†], XAVIER KESTELYN[‡], AND FRANCISCO CHINESTA[†]

Abstract. We present a nonlinear regression framework based on tensor algebra tailored to scarce data contexts. We exploit algebraic properties of a partial tensor product namely the m-tensor product, to leverage structured equations with separated variables. The proposed method combines kernel properties along with tensor algebra to prevent the curse of dimensionality and tackle approximations up to hundreds of parameters while avoiding the fixed point strategy. This formalism allows us to provide different regularization techniques fit for low amount of data with a high number of parameters while preserving well-known matrix-based properties. We demonstrate scaling on applications that are kept general but show robustness for engineering problems.

Key words. Multilinear algebra, Tensor format, High dimensional regression, Kernel-based regression

MSC codes. 15A23, 15A69, 65D40

1. Introduction. Engineering problems are often made dependent on parameters. In certain cases the number of those parameters might be reduced by identifying fewer latent variables but in certain scenarios the dependence has to be explicit. In that case even for problems with low latent dimensionality, making explicit use of those parameters becomes intractable. This issue is often found in optimization problems for design for instance. Manipulating a large number of parameters often becomes numerically intractable because of a wall of complexity coined *curse of dimensionality* [4]. This curse translates the exponential growth of models (*i.e.* the number of coefficients) and of the required amount of data with respect to the number of parameters. In contexts where data is scarce and costly, that curse makes models simply inaccessible.

Solutions proposed to circumvent this curse can rely on a modification of the grid required by the model, *e.g.* with a sparse grid [15], or with an adaptive selection of the pertinent points of that grid to retain [6]. A reduction of complexity can also be found by leveraging the structure of the equations. A common approach is to formulate the problem with separated variables. A prime example of this approach can be found in the works of Beylkin and Mohlenkamp [5]. The separated representation inherently relies on a tensorial factorization of the operators, often based on the use of Kronecker product based linear systems [2, 19]. This reliance on tensor representation can explain the more recent development of plenty of methods based on the tensor network formalisms, more specifically the Tensor-Train (TT) format [23] and overall gain of interest toward tensor methods [17, 12, 13].

A popular variable separation technique stemming from computational mechanics is the Proper Generalized Decomposition (PGD) [1] and its sparse grid based version, the s-PGD [14]. Those approaches rely on a Canonical Polyadic Decomposition (CPD) of the field that is learnt from one-dimensional modes, *i.e.* a rank-1 tensor is added at each loop on all axes. It resembles the CP-ALS method [17] and solves for each mode with a fixed-point strategy. This fixed-point approach implies a sequential method and may append modes late in the algorithm that serve only as correctives for early ones that were not optimal.

Many recent approaches leverage the TT-Format and its good scaling with high-dimensional problems. A framework that inspired this paper is that of MANDy [10]. In this work, Gelß et al. adapt the popular SINDy framework [7] to high-dimensional problems that remained a limitation for SINDy. To achieve good scaling, the authors built sparse TT cores from a choice of one-dimensional basis functions. The same authors later expanded the same methodology to image classification [16] that started to mix their tensorial framework with kernel interpretations to tackle efficiently higher dimensional problems without relying on the TT-SVD sequential algorithm. At the same time as Baddoo et al. with their fully kernel-based approach to dynamic model learning [3], Gelß et al. published the kernel Feature Space Approximation (kFSA) method that once again mixes TT-formulated feature maps to kernel methods to provide a high-dimensional regularization (and model reduction) technique. We further investigated the work of Engel et al. in signal processing [8] that kernelized the recursive least squares algorithm for online learning of signals. In their work, they provide nice proofs on the limitation of the number of retained signals and a bound with respect to ℓ_2 optimality provided by the kernel Principal Component Analysis (kPCA). The evaluation of distance in the feature space that defines their Almost Linearly Dependent (ALD) criterion is the same later used by Gelß and Baddoo in their respective works.

*Submitted to the editors DATE.

Funding: This work was funded by the ENSAM RTE Chair.

[†]ENSAM RTE Chair, PIMM, Arts et Metiers ParisTech, Paris, France (remi.cloarec@ensam.eu, sebastian.rodriguez_iturra@ensam.eu, francisco.chinesta@ensam.eu).

[‡]ENSAM RTE Chair, University of Lille, Arts et Metiers ParisTech, Centrale Lille, HEI, EA 2697-L2EP, Lille, France (xavier.kestelyn@ensam.eu).

With our framework, we keep the philosophy of structuring the equations that build the regression operator from a separated variables perspective. Considering the lack of data as a structural constraint, we isolate it in the structure of the proposed operator and leverage a *partial* tensor product. We consider it an algebraically minimalistic approach to generalize matrix-based regression. It allows us to preserve properties of usual matrix-based linear systems while avoiding the curse of dimensionality. By avoiding the fixed point strategy usually encountered in high dimensional contexts, we can produce an optimal result in the ℓ_2 sense from available data at a very low cost. We present the approach as *algebraically minimalistic* for its conceptual simplicity compared to tensor networks or abstract kernel feature spaces for instance. Working with tensor algebra and *then* identifying the kernel from the dot products also allows us to avoid the usual limitation of kernel methods to a few well-known kernels while leveraging their properties. For instance we are able to use the properties of kPCA. Furthermore, the sought minimalism allows us to avoid sparse operators that would be necessary in practice for computational efficiency in the case of tensor trains. The structure of the solution we build relates to the state of the art as a rank-wise unconstrained version of the sPGD.

We build our approach around a tensor formalism that provides the algebraic properties we leverage. We call that format *m-tensor* as it strongly relates to the unmanageable *matrix* that would be built naively to solve the problem. This is how we retain many of the desirable properties of the matrix-based approach in a fully factorized manner. It reveals that preserving the structure in higher order is what provides the great efficiency of tensor-based approaches. Those would otherwise be lost by the matrix-based structure. As it is tailored for regression operators, this formalism is not investigated as a potential data structure for high order problems.

The paper is organized as follows. In section 2, we set the problem of the least squares approximation of a scalar function in a general form. Section 3 builds the link between separable variables and tensor structure, this leads us to present common tensor operations that we will use to present our framework. The m-tensor framework is presented in the case of least squares regression in section 4. Section 5 provides different regularization techniques for m-tensor regression that we relate to kernel regression in section 6. We apply our method to demonstrate scalability and robustness on a polynomial benchmark with growing dimension and varying amount of data and then on two dynamical systems: Lorenz attractor and Kuramoto oscillators, in section 7. Section 8 draws conclusions and presents perspectives of future work.

Notations. We present the notations adopted in the remainder of the paper. Operation symbols are given with their definition. Working with high dimensional objects relies on burdensome notations in order to disambiguate definitions, therefore we try to strictly stick to conventions given here. Uppercase bold symbols (*e.g.* $\mathbf{T}$) are used for tensors. Uppercase symbols (*e.g.* M) are used for matrices. Lowercase bold symbols (*e.g.* $\mathbf{v}$) are used for vectors. The notation $[\cdot]_{ijk\dots}$ denotes the $ijk\dots$ -th element of the tensor in brackets, each index evolving along an axis of the tensor. Due to the high order nature of the object of the paper, indices are more frequently subscripted with axis number, *i.e.* elements of a tensor $\mathbf{T}$ of order N will be denoted as $[\mathbf{T}]_{i_1 i_2 \dots i_N}$.

2. Least squares approximation of a scalar function. In order to define a general approach, we define a domain $\mathcal{D} \subset \mathbb{R}^N$ as a tensor product of intervals in $\mathbb{R}$

$$\mathcal{D} = \bigotimes_{i=1}^N [a_i, b_i], \quad [a_i, b_i] \subset \mathbb{R}.$$

Now the aim is to approximate a scalar function f defined over that domain. We chose the form of approximation we seek by letting φ denote bases functions to be chosen, then the approximation has the form

$$(2.1) \quad \hat{f}(\mathbf{x}) = \sum_{i=1}^{\mathcal{N}} c_i \varphi_i(\mathbf{x}),$$

$$(2.2) \quad = \boldsymbol{\varphi}^T(\mathbf{x}) \mathbf{c},$$

where coefficients $\mathbf{c}$ are the *best fit* for the chosen bases functions. The number of terms $\mathcal{N}$ depends on the choice of bases as well. This formulation is meant to accomodate a large array of problems in a uniform manner.

Let $\mathcal{P}$ denote a set of m distinct samples in $\mathcal{D}$

$$\mathcal{P} = \{\mathbf{x}_k \in \mathbb{R}^N, \forall k, 1 \leq k \leq m\}.$$

Consider we are given the evaluation of f at each sample of $\mathcal{P}$. We denote $\mathbf{y} \in \mathbb{R}^m$ the vector defined as

$$[\mathbf{y}]_k = f(\mathbf{x}_k) \quad \forall k, 1 \leq k \leq m, \mathbf{x}_k \in \mathcal{P}.$$

Let $\Phi \in \mathbb{R}^{m \times \mathcal{N}}$ denote the evaluation of bases φ over the set $\mathcal{P}$. With $[\Phi]_k$ denoting the k -th row of Φ , we have

$$[\Phi]_k = \varphi^\top(\mathbf{x}_k) \quad \forall k, 1 \leq k \leq N, \mathbf{x}_k \in \mathcal{P}.$$

Then, the least squares solution $\hat{\mathbf{c}}$ is the one that, for any given bases φ , minimizes the squared ℓ_2 error with respect to $\mathbf{y}$:

$$(2.3) \quad \hat{\mathbf{c}} = \underset{\mathbf{c}}{\operatorname{argmin}} \sum_{k=1}^m \|\varphi^\top(\mathbf{x}_k) \mathbf{c} - [\mathbf{y}]_k\|_2^2.$$

This minimization problem admits an explicit solution based on whether Φ has linearly independent columns (2.4) or rows (2.5) that we build using the Moore-Penrose inverse formulation [24]:

$$(2.4) \quad \hat{\mathbf{c}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{y},$$

$$(2.5) \quad \hat{\mathbf{c}} = \Phi^\top (\Phi \Phi^\top)^{-1} \mathbf{y}.$$

Our choice is to use this widely used explicit formulation of least squares solution to determine the coefficients $\mathbf{c}$ of the approximation. Without any prior knowledge of f , the aim is then to define bases φ such that f is well approximated. To that end, one can chose to tailor the functions in order to minimize $\mathcal{N}$ or provide a rich enough set of bases so that all features of f can be captured. In what follows we chose the latter and show how our tensorial framework alleviates the curse of dimensionality within the regression approach.

3. Separable variables and tensor structure. A popular approach in high dimension is the separation of variables. In this setting, the bases functions φ are sought as products of one-dimensional functions. We denote those one-dimensional functions ψ . This leads us to define multidimensional bases φ as

$$\forall i, 1 \leq i \leq \mathcal{N}, \varphi_i(\mathbf{x}) = \prod_{j=1}^N \psi_{ij}([\mathbf{x}]_j).$$

Like all combinatorial problems, no standard approach would scale well in order to compute all combinations. For that reason we demonstrate the use of a convenient tensor framework. Let $\boldsymbol{\psi} \in \mathbb{R}^p$ denote the vector of p one dimensional basis functions deemed rich enough to represent the problem in one dimension given as

$$\boldsymbol{\psi}(x) = [\psi_1(x) \quad \psi_2(x) \quad \cdots \quad \psi_p(x)].$$

All combinations of one-dimensional bases would be found without redundancy in the elements of the tensor $\boldsymbol{\varphi}$ defined as

$$\boldsymbol{\varphi}(\mathbf{x}) = \bigotimes_{j=1}^N \boldsymbol{\psi}([\mathbf{x}]_j).$$

We note that $\boldsymbol{\varphi} \in \mathbb{R}^{p \times p \times \cdots \times p}$ is a rank-1 tensor of order N . Having the same p bases along each axis is an arbitrary choice made to alleviate notations.

Based on this representation, the formulation of the approximations of f in (2.1) can be written as the inner product

$$(3.1) \quad \hat{f}(\mathbf{x}) = \langle \boldsymbol{\varphi}(\mathbf{x}), \mathbf{C} \rangle.$$

Here $\mathbf{C} \in \mathbb{R}^{p \times p \times \cdots \times p}$ is an order N tensor of coefficients. The least squares coefficient tensor $\hat{\mathbf{C}}$ is the one minimizing the ℓ_2 error with respect to $\mathbf{y}$:

$$(3.2) \quad \hat{\mathbf{C}} = \underset{\mathbf{C}}{\operatorname{argmin}} \sum_{k=1}^m \|\langle \boldsymbol{\varphi}(\mathbf{x}_k), \mathbf{C} \rangle - [\mathbf{y}]_k\|_2^2.$$

If we were to unfold (vectorize) this tensor formulation, we would find the expression given for (2.1) with $\mathcal{N} = p^N$. This exponential growth with respect to the dimension N makes this kind of problem unmanageable for high dimensions, it is commonly known as the curse of dimensionality. Given this structure, we propose a tensor framework to efficiently build approximations in high dimensions.

We give a few reminders to work with tensors that allow us to introduce our framework. Tensors are high order objects and their use allows far more flexibility than the usual vector/matrix linear algebra. Their use allows to preserve structure that would be lost by flattening the problem in a matrix/vector formulation. The most important aspect is the ability to leverage factorized formulations for problems that would otherwise be inaccessible memory- and computationally-wise. We justify the given complexities in appendix section A.

3.1. Row-wise tensor product. We first define a row-wise tensor product. Further in this paper we refer to it as the *m-tensor product*. It generalizes the face splitting product [27] (transposed Khatri-Rao) the same way the tensor product generalizes the matrix Kroenecker product.

Our approach uses a similar treatment of the indices as the face-splitting product but conserves its multidimensional structure, therefore avoiding the index mapping burden. We define the m-tensor product of the two matrices $A \in \mathbb{R}^{m \times n_1}$ and $B \in \mathbb{R}^{m \times n_2}$, denoted $A \otimes B \in \mathbb{R}^{m \times n_1 \times n_2}$ as the *partial* tensor product preserving the number of rows of the input matrices. In indicial notation, it is defined as :

$$[A \otimes B]_{kij} = [A]_{ki} [B]_{kj}.$$

This product generalizes to an arbitrary number of input matrices, provided they all share the same number of rows. For a given set of matrices $\{\Psi_i \in \mathbb{R}^{m \times p_i}\}_{i=1}^N$, we build the order $N+1$ tensor $\Phi \in \mathbb{R}^{m \times p_1 \times p_2 \times \dots \times p_N}$ from the m-tensor product as

$$\begin{aligned} [\Phi]_{ki_1 i_2 \dots i_N} &= \left[\bigotimes_{j=1}^N \Psi_j \right]_{ki_1 i_2 \dots i_N}, \\ &= \prod_{j=1}^N [\Psi_j]_{ki_j}. \end{aligned}$$

From this definition, the face-splitting product relates readily as the mode-1 unfolding of the m-tensor product. We show further in this paper how well suited this tensor format is for multilinear regression. We refer to the matrices $\{\Psi_i\}_{i=1}^N$ as *cores* of the *m-tensor*.

We introduce the *r-dim* and *c-dim* of the resulting tensor as the dimensions of the *rows* and *columns* of this tensor respectively. In this setting we call "rows" the rank-1 tensors resulting from the row-wise tensor product. The mode-1 unfolding of this tensor, denoted $\Phi_{(1)}$, results in a dense matrix in $\mathbb{R}^{m \times p_1 p_2 \dots p_N}$. An important note is to be made of the structure of this unfolding. It is *exactly* equal to the face-splitting product of its cores defined using notations introduced by Slyusar [27] as

$$\Phi_{(1)} = \Psi_1 \square \Psi_2 \square \dots \square \Psi_N.$$

This can easily be demonstrated element-wise from both definitions. From this observation, we access properties of an otherwise inaccessible matrix operator $\Phi_{(1)}$ based on tricks allowed by a separated tensor representation.

3.2. Transposition. As is the case for arbitrary tensors, the transposition results in a reordering of the tensor's axes. For example with $\Phi \in \mathbb{R}^{m \times p_1 \times p_2 \times \dots \times p_N}$ built as the m-tensor product of cores $\{\Psi_j\}_{j=1}^N$, its transpose is defined by

$$\Phi^\top \in \mathbb{R}^{p_N \times p_{N-1} \times \dots \times p_1 \times m}, \text{ with } [\Phi^\top]_{i_N i_{N-1} \dots i_1 k} = \prod_{j=1}^N [\Psi_j]_{ki_j}.$$

3.3. Contractions. Contractions are operations that consist in summing axes of tensors, reducing its order. Any kind of contraction defined in the general case can be applied to the case of m-tensors. We specify here two kinds of contractions for m-tensors: the c-dim contraction and the r-dim contraction. We denote them by $\text{contract}_c(\cdot)$ and $\text{contract}_r(\cdot)$ respectively. Then we have for $\Phi \in \mathbb{R}^{m \times p_1 \times p_2 \times \dots \times p_N}$ the c-dim contraction defined as

$$\text{contract}_c(\Phi) \in \mathbb{R}^{p_1 \times p_2 \times \dots \times p_N}, \quad [\text{contract}_c(\Phi)]_{i_1 i_2 \dots i_N} = \sum_{k=1}^m [\Phi]_{ki_1 i_2 \dots i_N} \quad \forall i_j, 1 \leq i_j \leq p_j.$$

The r-dim contraction is defined as

$$\text{contract}_r(\Phi) \in \mathbb{R}^m, \quad [\text{contract}_r(\Phi)]_k = \sum_{i_1=1}^{p_1} \sum_{i_2=1}^{p_2} \dots \sum_{i_N=1}^{p_N} [\Phi]_{ki_1 i_2 \dots i_N} \quad \forall k, 1 \leq k \leq m.$$

The r-dim contraction acts on rank-1 tensors whose contraction is computed with complexity $\mathcal{O}(N(1+p))$ bringing the overall r-dim contraction to a computational cost of $\mathcal{O}(mN(1+p))$. This property demonstrated in appendix makes operations involving m-tensors r-dim contractions desirable. In the contrary we avoid c-dim contractions whose complexity is of $\mathcal{O}(mp^N)$. We denote $\text{contract}_{r,c}(\cdot)$ the full contraction of the tensor resulting in a scalar.

3.4. Hadamard product. The Hadamard product is a fundamental operation otherwise known as the element-wise product. The Hadamard product of two m-tensors can be computed in a fully factorized manner, therefore the scaling of its computation will not suffer the curse of dimensionality. Let $\Phi_1, \Phi_2 \in \mathbb{R}^{m \times p_1 \times p_2 \times \dots \times p_N}$ be m-tensors respectively based on cores $\{\Psi_i^1\}_{1 \leq i \leq N}$ and $\{\Psi_i^2\}_{1 \leq i \leq N}$.

$$\Phi_1 = \bigotimes_{i=1}^N \Psi_i^1, \quad \Phi_2 = \bigotimes_{i=1}^N \Psi_i^2.$$

Elements of their Hadamard product are defined as

$$\begin{aligned} [\Phi_1 \odot \Phi_2]_{ki_1 i_2 \dots i_N} &= [\Phi_1]_{ki_1 i_2 \dots i_N} [\Phi_2]_{ki_1 i_2 \dots i_N}, \\ &= \prod_{j=1}^N [\Psi_j^1]_{ki_j} [\Psi_j^2]_{ki_j}. \end{aligned}$$

The complexity of the Hadamard product of two m-tensors is $\mathcal{O}(Nmp)$ by letting $\forall i, p_i = p$ for ease of notation.

3.5. Inner product and norm. The inner product of two m-tensors leverages the efficiency of the r-dim contraction that we defined earlier. Let $\Phi_1, \Phi_2 \in \mathbb{R}^{m \times p_1 \times p_2 \times \dots \times p_N}$ be the m-tensors defined as in the previous section. Their inner product is defined as

$$\begin{aligned} \langle \Phi_1, \Phi_2 \rangle &= \sum_{k=1}^m \sum_{i_1=1}^{p_1} \sum_{i_2=1}^{p_2} \dots \sum_{i_N=1}^{p_N} [\Phi_1]_{ki_1 i_2 \dots i_N} [\Phi_2]_{ki_1 i_2 \dots i_N}, \\ &= \text{contract}_{r,c}(\Phi_1 \odot \Phi_2). \end{aligned}$$

The complexity for this inner product is $\mathcal{O}(m(1 + N(1 + p)))$ considering $\forall i, p_i = p$ for ease of notation. The ℓ_2 norm of an m-tensor inherits the efficiency highlighted above. We define it for $\Phi \in \mathbb{R}^{m \times p_1 \times p_2 \times \dots \times p_N}$ as

$$\begin{aligned} \|\Phi\|_2 &= \sqrt{\langle \Phi, \Phi \rangle}, \\ &= \sqrt{\sum_{k=1}^m \sum_{i_1=1}^{p_1} \sum_{i_2=1}^{p_2} \dots \sum_{i_N=1}^{p_N} [\Phi]_{ki_1 i_2 \dots i_N}^2}, \\ &= \sqrt{\text{contract}_{r,c}(\Phi^{\odot 2})}. \end{aligned}$$

3.6. N-way contracted product. Operations with tensors allow to define the sums and product depending on the aim of the operation. We leverage a N -way contracted product. We call it the m -product and denote it $\bowtie$. Let $\Phi_1 \in \mathbb{R}^{m_1 \times p_1 \times p_2 \times \dots \times p_N}$ and $\Phi_2 \in \mathbb{R}^{m_2 \times p_1 \times p_2 \times \dots \times p_N}$ be m-tensors respectively based on cores $\{\Psi_i^1\}_{1 \leq i \leq N}$ and $\{\Psi_i^2\}_{1 \leq i \leq N}$. Provided that both tensors have the same order and same row dimensions, we can build an element of $\Phi^1 \bowtie (\Phi^2)^\top \in \mathbb{R}^{m_1 \times m_2}$ as the inner product of order $N - 1$ rank-1 tensors. The overall computation results in the Hadamard product of matrix products of corresponding cores of the input m-tensors:

$$(3.3) \quad \Phi_1 \bowtie \Phi_2^\top = \bigodot_{i=1}^N \Psi_i^1 (\Psi_i^2)^\top.$$

Considering, for ease of notation that $m_1 = m_2 = m$ and $\forall i, 1 \leq i \leq N, p_i = p$, the complexity of the m-product is $\mathcal{O}(m^2 p N)$. This complexity scales only linearly with dimension N and is dominated by m^2 . The m-product of arbitrary tensors with the same dimensions would result in a complexity $\mathcal{O}(m^2 p^N)$.

4. M-Tensor least squares regression. In this section, we demonstrate the use of the m-tensor formalism to efficiently compute a multidimensional least squares regression. The aim is to make the tensor least squares (3.2) manageable in arbitrarily high dimensions. Provided the set of sampled parameters $\mathcal{P} = \{\mathbf{x}_k\}_{k=1}^m$ and corresponding evaluations of f in a vector $\mathbf{y}$, we define tensor basis functions as

$$\varphi(\mathbf{x}) = \bigotimes_{i=1}^N \psi([\mathbf{x}]_i), \quad \text{with} \quad \psi(x) = [\psi_1(x) \quad \psi_2(x) \quad \dots \quad \psi_p(x)].$$

This structure amounts to combining one-dimensional bases ψ for all input parameters. This gives us a large approximation capability, unreachable with a matrix-based regression. We remark that by stacking the rank-1 tensors φ , we actually build an m-tensor. To that end, we provide the following core definition for axis i , *i.e.* built of one-dimensional basis functions of the i -th components of each sample in $\mathcal{P}$.

$$\Psi_i = \begin{bmatrix} \psi_1([\mathbf{x}_1]_i) & \psi_2([\mathbf{x}_1]_i) & \cdots & \psi_p([\mathbf{x}_1]_i) \\ \psi_1([\mathbf{x}_2]_i) & \psi_2([\mathbf{x}_2]_i) & \cdots & \psi_p([\mathbf{x}_2]_i) \\ \vdots & \vdots & \ddots & \vdots \\ \psi_1([\mathbf{x}_m]_i) & \psi_2([\mathbf{x}_m]_i) & \cdots & \psi_p([\mathbf{x}_m]_i) \end{bmatrix}.$$

That definition allows us to define the m-tensor Φ as

$$(4.1) \quad \Phi = \bigotimes_{i=1}^N \Psi_i.$$

We can now reformulate the regression problem in m-tensorial setting. We seek the regression coefficients $\mathbf{C} \in \mathbb{R}^{p \times p \times \cdots \times p}$ such that

$$(4.2) \quad \Phi \ltimes \mathbf{C} = \mathbf{y}.$$

A least squares solution $\hat{\mathbf{C}}$ to this problem can be built explicitly based on the general pseudoinverse formulation. Provided that Φ has linearly independent rows, we have the solution

$$(4.3) \quad \hat{\mathbf{C}} = \Phi^\top (\Phi \ltimes \Phi^\top)^{-1} \mathbf{y}.$$

We cannot use this formulation as is. The dimensionality of $\hat{\mathbf{C}}$ would require to store p^N elements, which rapidly become unmanageable with growing N . Instead, by leveraging properties of the m-tensor product, we give the approximation of f as

$$\hat{f}(\mathbf{x}) = \varphi(\mathbf{x}) \ltimes \Phi^\top (\Phi \ltimes \Phi^\top)^{-1} \mathbf{y}.$$

This expression purposefully explicits the use of the m-product to show how it results in only inner products. In fact, by denoting

$$P = \Phi \ltimes \Phi^\top \in \mathbb{R}^{m \times m},$$

and defining

$$\mathbf{z} = P^{-1} \mathbf{y} \in \mathbb{R}^m.$$

We give the online computation of $\hat{f}$ requiring only the computation of the vector made from $\varphi(\mathbf{x}) \ltimes \Phi^\top \in \mathbb{R}^m$ and its inner product with $\mathbf{z}$:

$$\hat{f}(\mathbf{x}) = \langle \varphi(\mathbf{x}) \ltimes \Phi^\top, \mathbf{z} \rangle.$$

This computation results in a complexity of $\mathcal{O}(m(1 + Np))$. With a standard matrix-based approach, even storing the coefficients would not be possible, scaling exponentially with N with a rate at least p^N .

Remark 4.1. The solution, although mathematically sound, proves numerically unstable because of the inversion of P . For that reason, we highlight the fact that by construction, P is symmetric positive definite and therefore admits a Cholesky factorization. We give that decomposition as

$$P = LL^\top,$$

with $L \in \mathbb{R}^{m \times m}$ a lower triangular matrix. Computation of $\mathbf{z}$ can then be done through a two step procedure. We first solve for an intermediate result $\bar{\mathbf{z}}$ by upward backsubstitution

$$\bar{\mathbf{z}} = L^{-1} \mathbf{y}.$$

Then $\mathbf{z}$ is solved for by downward backsubstitution with

$$\mathbf{z} = (L^\top)^{-1} \bar{\mathbf{z}}.$$

5. Regularization of m-tensor regression. Mimicking a standard (matrix-based) regression *but* allowing so many combinations of one-dimensional bases to express the problem makes the m-tensor regression prone to overfitting and leads to severely ill-conditioned systems. Therefore we address the crucial ability to regularize it. Generally the aim of regularization is to enhance the conditioning of a matrix. A matrix Φ with rank r has a condition number κ defined as

$$(5.1) \quad \kappa(\Phi) = \frac{\sigma_1}{\sigma_r}.$$

Here σ_1 denotes the largest singular value of Φ and σ_r its lowest nonzero singular value. A matrix is known to be *ill-conditioned* if κ is very large. The strength of m-tensor regression is the amount of bases it uses to approximate the problem, it also plagues it with poor conditioning, thus requiring the following work on regularization.

Regularized regression will take the form of penalizations added to the least square minimization problem (2.3). Let $\mathcal{R}$ denote a penalization term, the new minimization problem is formulated as

$$(5.2) \quad \hat{\mathbf{c}}_{\mathcal{R}} = \underset{\mathbf{c}}{\operatorname{argmin}} \left(\|\Phi \mathbf{c} - \mathbf{y}\|_2^2 + \mathcal{R}(\mathbf{c}, \boldsymbol{\lambda}) \right).$$

The penalization term $\mathcal{R}$ is intended to privilege certain forms of solution in the minimization. It acts based on the terms of $\mathbf{c}$ and a *regularization parameter* (or set of parameters) $\boldsymbol{\lambda}$. We suggest ways to act in a memory efficient manner. The case of m-tensor regression yields a minimization problem excessively similar to the classical one:

$$(5.3) \quad \hat{\mathbf{C}}_{\mathcal{R}} = \underset{\mathbf{C}}{\operatorname{argmin}} \left(\|\Phi \ltimes \mathbf{C} - \mathbf{y}\|_2^2 + \mathcal{R}(\mathbf{C}, \boldsymbol{\lambda}) \right).$$

5.1. Tikhonov regularization. Regular matrix-based Tikhonov regularization solves for (5.2) with

$$\mathcal{R}(\mathbf{c}, \lambda) = \lambda \|\mathbf{c}\|_2.$$

This leads to a minimization of the ℓ_2 norm of $\mathbf{c}$. This minimization problem admits an explicit solution involving the SVD of the matrix Φ that we denote

$$\Phi = U \Sigma V^{\top}.$$

Then the ridge regression solution with parameter λ is

$$\hat{\mathbf{c}}_2 = V D_{\lambda} U^{\top} \mathbf{y}.$$

Where D_{λ} is a diagonal matrix with

$$(5.4) \quad D_{ii} = \frac{\sigma_i}{\sigma_i^2 + \lambda^2}.$$

One can see how this elevates the values of the lowest singular values, hence limiting the rise of κ in equation (5.1). In fact the effect of Tikhonov regularization is to level up singular values of Φ below the set threshold. An equivalent solution is given as

$$(5.5) \quad \hat{\mathbf{c}}_2 = \Phi^{\top} (\Phi \Phi^{\top} + \lambda^2 \mathbb{I})^{-1} \mathbf{y}.$$

We generalize this approach to m-tensor regression by leveraging the m-tensor SVD. Consider the m-tensor formulation (5.3), with regularization term

$$\mathcal{R}(\mathbf{C}, \lambda) = \lambda \|\mathbf{C}\|_2.$$

Analogously to the matrix case, we can explicitly give the regularized solution as

$$\hat{\mathbf{C}}_2 = \Phi^{\top} (\Phi \ltimes \Phi^{\top} + \lambda^2 \mathbb{I})^{-1} \mathbf{y}.$$

Then the same diagonal matrix D_{λ} as equation (5.4) can be built by noticing that we can access equivalent left singular vectors U and singular values Σ from the SVD of $\Phi \ltimes \Phi^{\top}$. The regularized coefficients $\hat{\mathbf{C}}_2$ can be computed as

$$\begin{aligned} \hat{\mathbf{C}}_2 &= \Phi^{\top} (\Phi \ltimes \Phi^{\top} + \lambda^2 \mathbb{I})^{-1} \mathbf{y}, \\ &= \Phi^{\top} (U \Sigma^2 U^{\top} + \lambda^2 \mathbb{I})^{-1} \mathbf{y}, \\ &= \Phi^{\top} U D_{\lambda} U^{\top} \mathbf{y}. \end{aligned}$$

The approximation of f with Tikhonov regularization is then given by evaluating the inner product

$$(5.6) \quad \hat{f}_2(\mathbf{x}) = \langle \boldsymbol{\varphi}(\mathbf{x}) \ltimes \boldsymbol{\Phi}^\top, \mathbf{z}_2 \rangle.$$

In this expression $\mathbf{z}_2$ is defined as

$$\mathbf{z}_2 = UD_\lambda U^\top \mathbf{y}.$$

This approaches' complexity is dominated by the construction of the m-product $\boldsymbol{\Phi} \ltimes \boldsymbol{\Phi}^\top$ (in $\mathcal{O}(m^2 p N)$) and that of its SVD (in $\mathcal{O}(m^3)$). Inference cost, *i.e.* the online evaluation of the approximation given in (5.6), is of complexity $\mathcal{O}(mpN)$.

5.2. Spectral truncation. In the matrix-based regression context, spectral truncation aims at improving the condition number of the regression operator by simply removing (setting to zero) the singular values below a set threshold τ . This is equivalent to solving for (5.2) with a penalization on the nuclear norm of $\boldsymbol{\Phi}$, *i.e.* the sum of its singular values. Let $\boldsymbol{\Phi} = U\Sigma V^\top$ and the least squares regression coefficients be defined as

$$\hat{\mathbf{c}} = V\Sigma^{-1}U^\top \mathbf{y}.$$

The spectral truncation regularization consists in replacing Σ by a matrix D_τ to get the regularized coefficients as

$$\hat{\mathbf{c}}_* = UD_\tau V^\top \mathbf{y} \quad \text{where} \quad [D_\tau]_{ii} = \begin{cases} \sigma_i & \text{if } \sigma_i \geq \tau \\ 0 & \text{else} \end{cases}.$$

We generalize this approach to the m-tensor regression by, once again, leveraging the SVD of $P = \boldsymbol{\Phi} \ltimes \boldsymbol{\Phi}^\top$. We are able to truncate the rank so as to regularize the regression. We select the r first singular values, deemed sufficient to represent the system by comparing them to the input threshold τ . We denote U_r the matrix built from only the first r left singular vectors of P . The regularized coefficients are then given by

$$\hat{\mathbf{C}}_* = \boldsymbol{\Phi}^\top U_r U_r^\top P^{-1} \mathbf{y}.$$

With that approach, we project the least squares regression onto the span only of the r first left singular vectors. This approach is chosen so as to keep using the inner products of rank-1 tensors. The resulting approximation is computed as the inner product

$$\hat{f}_*(\mathbf{x}) = \langle \boldsymbol{\varphi}(\mathbf{x}) \ltimes \boldsymbol{\Phi}^\top, \mathbf{z}_* \rangle.$$

Where $\mathbf{z}_*$ is defined as

$$\mathbf{z}_* = U_r U_r^\top P^{-1} \mathbf{y}.$$

This approach is numerically stable, leverages previously required data and avoid full inversion. It results in a complexity dominated once again by the computation of $\boldsymbol{\Phi} \ltimes \boldsymbol{\Phi}^\top$ and its SVD for the same inference cost as for the Tikhonov regularization. A randomized version of the SVD may be leveraged to speed up its computation if the rank is a given input.

5.3. ALI regularization. In this section we consider a regularization technique that aims at enhancing the conditioning of the system as well as improving the efficiency of the approximation. We consider a subset of $\tilde{m} < m$ rows of $\boldsymbol{\Phi}$, corresponding to a subsample $\tilde{\mathcal{P}} \subset \mathcal{P}$ that can approximate all rows of $\boldsymbol{\Phi}$ up to a given tolerance ε . The tensor's rows are therefore considered Almost Linearly Dependent (ALD) [8] on this subset. We denote $\tilde{\boldsymbol{\Phi}}$ the tensor made only of those $\tilde{m}$ Almost Linearly Independent (ALI) rows of $\boldsymbol{\Phi}$, then we have by denoting $[\boldsymbol{\Phi}]_k$ the k -th row of $\boldsymbol{\Phi}$

$$\forall k, 1 \leq k \leq m, \quad \delta_k = \min_{\mathbf{w} \in \mathbb{R}^{\tilde{m}}} \left\| [\boldsymbol{\Phi}]_k - \mathbf{w}^\top \tilde{\boldsymbol{\Phi}} \right\|_2^2 < \varepsilon.$$

If the row subsample of size $\tilde{m}$ is optimal with respect to the criterion ε , then there exists a matrix $W \in \mathbb{R}^{m \times \tilde{m}}$ such that

$$\left\| \boldsymbol{\Phi} - W\tilde{\boldsymbol{\Phi}} \right\|_2^2 < \varepsilon \quad \text{i.e.} \quad \boldsymbol{\Phi} \approx W\tilde{\boldsymbol{\Phi}}.$$

From that decomposition we define the symmetric positive definite matrix

$$\tilde{P} = \tilde{\boldsymbol{\Phi}} \ltimes \tilde{\boldsymbol{\Phi}}^\top.$$

Then we build the approximation

$$\tilde{f}(\mathbf{x}) = \langle \boldsymbol{\varphi}(\mathbf{x}) \tilde{\boldsymbol{\Phi}}, \tilde{\mathbf{z}} \rangle,$$

where $\tilde{\mathbf{z}}$ is defined as

$$\tilde{\mathbf{z}} = \tilde{P}^{-1}\mathbf{y}.$$

With this formulation, we are able to leverage the Cholesky factorization of $\tilde{P} = \tilde{L}\tilde{L}^\top$ and compute $\tilde{\mathbf{z}}$ from a double backsubstitution. The computational gain can be massive at the online stage if $\tilde{m} \ll m$. We evaluated the complexity of the least squares approximation inference at $\mathcal{O}(mpN)$. With the ALI regularization, we have a complexity of $\mathcal{O}(\tilde{m}pN)$. Even small gains in this formulation far outperform all previous approaches at a relatively small cost in terms of error.

Indeed we inherit a bound with respect to the spectral truncation approximation. Engel et al. [8] gave a bound on the span of the selected rows of the ALI decomposition of Φ with respect to the optimal decomposition in the least squares sense, *i.e.* the SVD. By defining the projector $\Pi_{\tilde{\Phi}}$ as

$$\Pi_{\tilde{\Phi}} = \tilde{\Phi}^\top (\tilde{\Phi} \times \tilde{\Phi}^\top) \tilde{\Phi},$$

we have

$$\frac{1}{m} \sum_{i=1}^m \|(\text{Id} - \Pi_{\tilde{\Phi}}) \varphi^\top(\mathbf{x}_i)\|_2^2 \leq \varepsilon.$$

We give the full algorithm to obtain an optimal or a greedy formulation of the ALI regression in appendix section B. This regression is similar to the formulation of s-PGD [14]. We share the non-orthogonality of the retained rank-1 tensor *modes* in $\tilde{\Phi}$ and very light formulation, especially well suited for inverse problems or surrogate evaluation.

6. Kernel regression interpretation. In this section we make a few remarks on the parallel between our approach and the kernel regression framework. In practice, we manage to always leverage the high efficiency of the dot product of rank-1 tensors. Therefore we avoid the usual kernel caveat. That is, we do not need to select a type of kernel or combine from a limited set of kernels, instead we directly work on the mapping with a tensor product structure. Usually the kernel literature struggles to move away from the few well known kernels: gaussian, polynomial or trigonometric.

We leverage the tensor product's ability to augment dramatically the row dimensions. Therefore we can always consider the linearly independent rows pseudoinverse of the m-tensor. In that case, we make $P = \Phi \times \Phi^\top$ appear in the pseudoinverse formulation so that only dot products are needed. That way, an element of P is given by the inner product $\langle \varphi(\mathbf{x}_i), \varphi(\mathbf{x}_j) \rangle$, considering the mapping

$$\varphi : \begin{cases} \mathbb{R}^N & \longrightarrow \mathbb{R}^{p \times p \times \dots \times p} \\ \mathbf{x} & \longmapsto \bigotimes_{i=1}^N \psi([\mathbf{x}]_i) \end{cases},$$

we can derive a kernel function k defined as the inner product

$$k : \begin{cases} \mathbb{R}^N \times \mathbb{R}^N & \longrightarrow \mathbb{R} \\ (\mathbf{x}, \mathbf{x}') & \longmapsto \langle \varphi(\mathbf{x}), \varphi(\mathbf{x}') \rangle \end{cases}.$$

Therefore we have the insurance to build a kernel that verifies Mercer's theorem [22]. There is a direct equivalence between the projector $P = \Phi \times \Phi^\top$ and the kernel matrix K defined with entries

$$[K]_{ij} = k(\mathbf{x}_i, \mathbf{x}_j).$$

Usual kernel approaches avoid defining the mapping, whereas we manage to leverage it in combination with the rank-1 tensor dot product to remain efficient with flexibility in the kernel definition. The efficiency of the rank-1 tensor dot product is a direct equivalence of the so-called *kernel trick* to our case.

Our method is inspired by works from Gelß et al. [9, 16], Baddoo et al. [3] and before that Engel et al. [8]. We obviously rely on the precursor works of Schölkopf et al. [26, 25] to derive properties of kernel methods and apply them to this tensor-based approach. From those observations we managed to derive powerful properties and methods, making the use of m-tensors a salient tool to solve high dimensional regression problems.

7. Numerical examples. In this section we present several numerical examples of use for the m-tensor regression framework. The aim is to demonstrate how easy the m-tensors are to use and their efficiency to resolve high dimensional problems.

7.1. Rosenbrock function approximation. We first demonstrate the scaling in complexity of the approach. To that end we build a regression of the Rosenbrock function generalized from 20 input parameters to 300: $N \in \{20, 50, 100, 200, 300\}$. The implementation is given step-by-step and results are analyzed. We chose the Rosenbrock function for ease of implementation and its arbitrary definition with respect to the input's dimension. The Rosenbrock function is usually defined over the domain $\mathcal{D} = [-5, 10]^{\otimes N}$ in dimension N as

$$f(\mathbf{x}) = \sum_{i=0}^{N-1} 100 ([\mathbf{x}]_{i+1} - [\mathbf{x}]_i^2)^2 + ([\mathbf{x}]_i - 1)^2.$$

The aim is to provide an approximation to that function of the shape

$$\hat{f}(\mathbf{x}) = \sum_{\mathcal{I}} c_{\mathcal{I}} \varphi_{\mathcal{I}}(\mathbf{x}).$$

Where $\mathcal{I}$ is a multi-index and basis functions φ are defined as products of one-dimensional functions. We explicit the tensor form and chose a polynomial mapping up to degree 4 defined by

$$\varphi(\mathbf{x}) = \bigotimes_{i=1}^N [1 \quad [\mathbf{x}]_i \quad [\mathbf{x}]_i^2 \quad [\mathbf{x}]_i^3 \quad [\mathbf{x}]_i^4].$$

Which results in seeking an order N coefficient tensor $\mathbf{C} \in \mathbb{R}^{5 \times 5 \times \dots \times 5}$ so that the approximation is given as a N -way contracted product

$$\hat{f}(\mathbf{x}) = \varphi(\mathbf{x}) \times \mathbf{C}.$$

We consider that for a dimension N we are given a latin hypercube sampling (LHS) of m points over the definition domain $\mathcal{D}$. We will consider a linear scaling of the sampling with respect to the dimension to demonstrate the efficiency of the approach in massively underdetermined cases, *i.e.* $m = \alpha N$. We will evaluate the efficiency and accuracy of the approach for varying $\alpha \in \{5, 10, 20, 30, 40\}$. We denote $\mathcal{P} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$ the LHS and $\mathbf{y}$ the evaluation of the Rosebrock function on that sampling:

$$\forall k, 1 \leq k \leq m, \quad [\mathbf{y}]_k = f(\mathbf{x}_k).$$

We build the m-tensor from the definition of φ and the given samples $\mathcal{P}$. To do so, we simply evaluate the *core* matrices. In this case, those are Vandermonde matrices given for each input variable

$$\forall i, 1 \leq i \leq N, \quad \Psi_i = \begin{bmatrix} 1 & [\mathbf{x}_1]_i & [\mathbf{x}_1]_i^2 & [\mathbf{x}_1]_i^3 & [\mathbf{x}_1]_i^4 \\ 1 & [\mathbf{x}_2]_i & [\mathbf{x}_2]_i^2 & [\mathbf{x}_2]_i^3 & [\mathbf{x}_2]_i^4 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & [\mathbf{x}_m]_i & [\mathbf{x}_m]_i^2 & [\mathbf{x}_m]_i^3 & [\mathbf{x}_m]_i^4 \end{bmatrix}.$$

Given those matrices, the m-tensor is fully defined and we can readily build our approximation. We denote the m-tensor Φ defined as

$$\Phi = \bigotimes_{i=1}^N \Psi_i.$$

7.1.1. Least squares and Thikonov regularization. We give the resolution procedure for Thikonov regularization and the least squares as a special case thereof. We define the matrix $P_\lambda \in \mathbb{R}^{m \times m}$ with regularization parameter λ as

$$P_\lambda = \Phi \times \Phi^\top + \lambda^2 \text{Id}.$$

This construction's complexity scales as $\mathcal{O}(m^2 N)$ which, considering a linear scaling of the sample size with the dimension becomes $\mathcal{O}(\alpha^2 N^3)$. We then need solve for $\mathbf{z}_2 \in \mathbb{R}^m$ with

$$P_\lambda \mathbf{z}_2 = \mathbf{y}.$$

Since P is full, this will scale as $\mathcal{O}(m^3)$, *i.e.* $\mathcal{O}(\alpha^3 N^3)$ with a linear scaling of the sampling size. In fact stability can be gained by computing the Cholesky decomposition of P_λ and then solve with a double backsubstitution, which does not impact complexity scaling. Approximation is then readily computed online as

$$\hat{f}_2(\mathbf{x}) = \langle \varphi(\mathbf{x}) \times \Phi^\top, \mathbf{z}_2 \rangle.$$

This computation's complexity is dominated by the N -way contracted product $\varphi(\mathbf{x}) \times \Phi^\top$. For each new input $\mathbf{x}$, computation has a complexity $\mathcal{O}(mN)$. Still considering a linear scaling of the sampling with dimension it yields an inference cost of $\mathcal{O}(\alpha N^2)$.

7.1.2. Spectral truncation regularization. Regularization by spectral truncation is achieved by computing first the matrix $P \in \mathbb{R}^{m \times m}$ defined as

$$P = \Phi \times \Phi^\top.$$

Just as in the least squares case, this computation scales as $\mathcal{O}(m^2 N)$. The spectral truncation requires computation of its SVD:

$$P = U \Sigma^2 U^\top.$$

This computation scales as $\mathcal{O}(m^3)$. This cost can be minimized by computing a randomized version of the SVD if the sought rank is predicted a priori. Once the rank r of the truncation is selected, U and Σ are truncated (which we denote by a subscript r) and we can compute $\mathbf{z}_*$ as

$$\mathbf{z}_* = U_r U_r^\top P^{-1} \mathbf{y}.$$

The online computation is then given by the inner product

$$\hat{f}_*(\mathbf{x}) = \langle \varphi(\mathbf{x}) \times \Phi^\top, \mathbf{z}_* \rangle.$$

There is no difference in computational complexity at the online stage when compared with the least squares.

7.1.3. ALI regularization. We consider the ALID algorithm provided in appendix to provide the approximation of Φ given by

$$\Phi \approx W \tilde{\Phi}.$$

We have seen that $\tilde{\Phi}$ consists in a subset of $\tilde{m} < m$ rows of Φ . For regression purposes, we do not need to compute the weights matrix W but we preserve the mask that selects the rows of Φ and apply the same onto the evaluation vector $\mathbf{y}$ to obtain $\tilde{\mathbf{y}} \in \mathbb{R}^{\tilde{m}}$. Instead of considering P as previously defined, we consider

$$\tilde{P} = \tilde{\Phi} \times \tilde{\Phi}^\top.$$

From this $\tilde{m} \times \tilde{m}$ matrix, we solve for $\tilde{\mathbf{z}}$ in

$$\tilde{P} \tilde{\mathbf{z}} = \tilde{\mathbf{y}}.$$

We are then able to build the online approximation as

$$\tilde{f}(\mathbf{x}) = \langle \varphi(\mathbf{x}) \times \tilde{\Phi}, \tilde{\mathbf{z}} \rangle.$$

The specifics of the algorithm for ALI regularization are given in appendix both for the greedy and the optimal approach. At this stage we consider them as complete algorithms as we did for the SVD. In the greedy case this computation is bounded (by far) by $\mathcal{O}(m^2 N)$, *i.e.* $\mathcal{O}(\alpha^2 N^3)$ considering linear scaling of the sampling with the dimension.

At the online stage complexity is reduced to $\mathcal{O}(\tilde{m} N)$. The inference time will therefore be dependent on the number of selected rows but will always be bounded by the complexity of precedently introduced approaches. The optimal ALID performance is not investigated and the curious reader should read the work of Gelß et al. [9] for further details on an analogous method based on TT-format.

7.1.4. Results. In the case of the Rosenbrock function we obtain errors within 10% and struggle more to tune hyperparameters at low dimension than to demonstrate the performance and scaling of the method in high dimensions. We plot surface results for models built for all combinations of dimension $N \in \{20, 50, 100, 200, 300\}$ and sample size per axis $\alpha \in \{5, 10, 20, 30, 40\}$. All models are built on a LHS, therefore we build them 3 times and plot an average to account for representativity issues of the sampling. There is no significant variability observed throughout samplings. The aim is to make complexity tendencies appear clearly.

The first observed quantity is the *unit inference time* in figure 7.1. It corresponds to the average time to compute a solution using the approximation we built. Results are plotted for the least squares model and the ALI regression model. The inference time for the spectral truncation model is the same as for the least squares. Those plots demonstrate the ability of the ALI regression model to fully linearize the cost with respect to the dimension. Scaling of the least squares appears overall quadratic because the linear scaling of the sampling size on the dimension cumulates with the linear scaling of the inference time on the dimension: $\mathcal{O}(mpN)$ with $m = \alpha N$ becomes $\mathcal{O}(\alpha N^2)$.

Figure 7.2 presents the relative ℓ_2 error of our models on a LHS of $3m$ unseen points. We notice that errors are worse for lower dimensions, where hyperparameters are harder to chose and models appear more sensitive. In higher dimensions, all models perform at about 2% error. The low convergence rate of the error with the number of training samples is inherited by the linear scaling of the sampling whereas the *ideal sampling* (interpolative) would

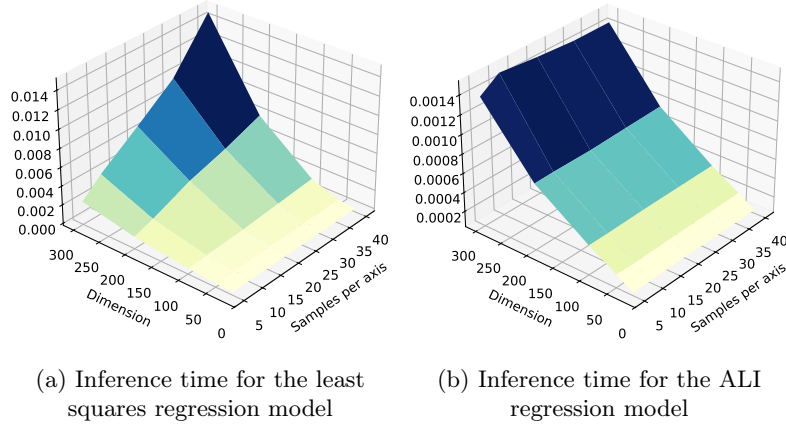


Figure 7.1: Evolution of the unit inference time [s] for the least squares and ALI regularization models in function of the number of samples per axis and the input dimension.

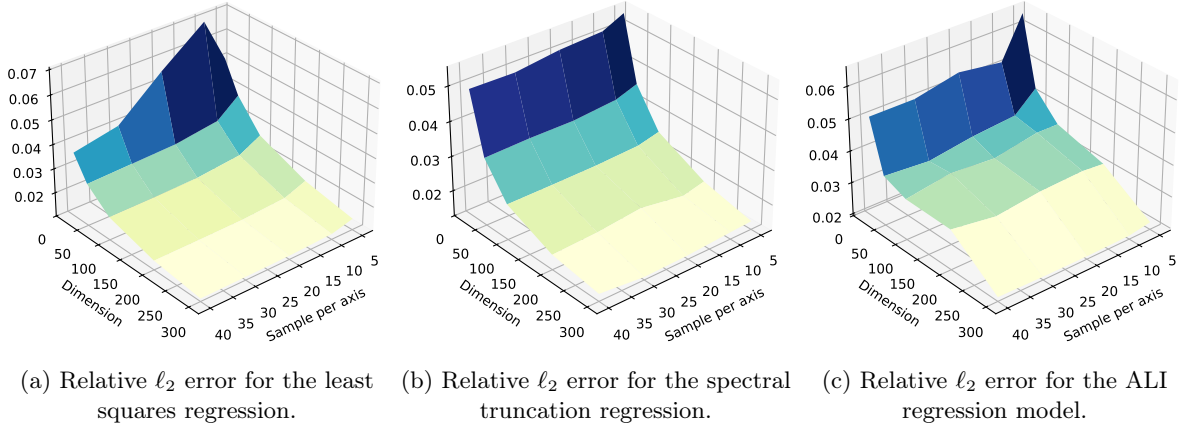


Figure 7.2: Evolution of the relative ℓ_2 error for the least squares, spectral truncation and ALI regularization models on a LHS of unseen points in function of the number of samples per axis and the input dimension.

be scaling exponentially. Therefore we suspect we must be at an extremely localized point on a larger convergence curve that would require too many points to build but starts to shape out at dimension $N = 20$.

Figure 7.3 presents the *offline* construction cost of the models. Those costs are assumed only once. Each method being quite different at this stage, results vary significantly. In the least squares case, we have to build $P = \Phi \times \Phi^\top$ which reaches up to $\sim 300s$ for $N = 300$ and $\alpha = 40$, to which is added the inversion of P to build $\mathbf{z}_2$. We observe that the algorithm is dominated by the cubic scaling on N inherited by both stages from $m = \alpha N$, and the cubic scaling on m itself due to the inversion stage: $m^3 = \alpha^3 N^3$. That results is observed again for the spectral truncation procedure with more overhead due to the SVD of P , whose scaling is also $\mathcal{O}(m^3)$. That added complexity more than tripples the time of the least squares procedure at $N = 300$ and $\alpha = 40$ to reach $\sim 1600s$.

The greedy ALI regression construction demonstrates the best scaling, apparently linear with α and therefore quadratic only with N . The incremental size of the system at each step of the algorithm makes it hard to exactly estimate the degree of the scaling on N but we can state that the upper bound provided before as $\mathcal{O}(m^2 N)$ *i.e.* $\mathcal{O}(\alpha^2 N^3)$ is far from being reached in the studied domain.

7.2. Prediction of dynamical systems in time. In the context of dynamical systems, regression is used to predict the evolution in time of the system. If the state is denoted $\mathbf{x}(t) \in \mathbb{R}^N$, then there exists a law f that

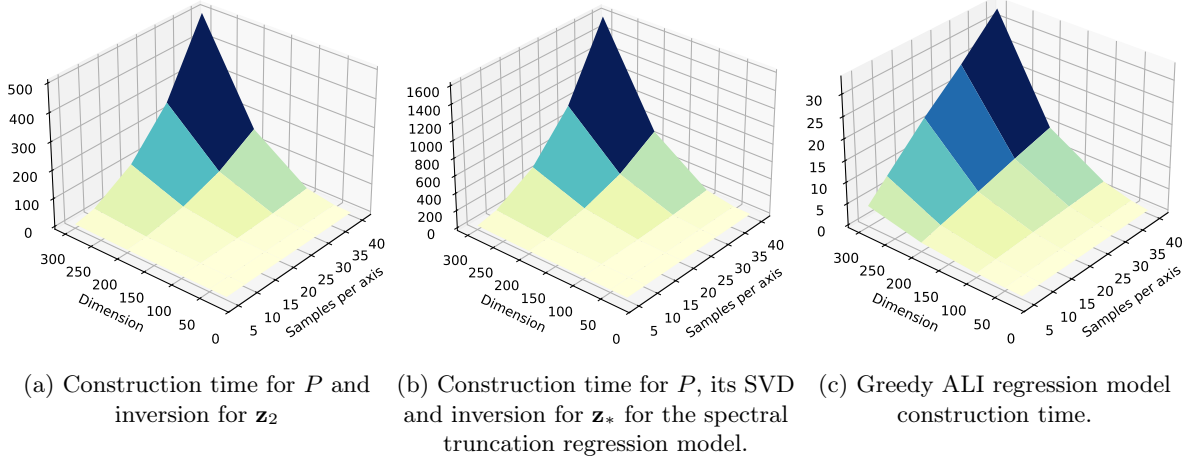


Figure 7.3: Evolution of the model construction times [s] for the least squares, spectral truncation and ALI regularization models in function of the number of samples per axis and the input dimension.

describes the variation of the state in time. This results in the general form

$$\frac{\partial \mathbf{x}}{\partial t} = f(\mathbf{x}).$$

The function f might be nonlinear. The aim is to approximate at best this function in order to be able to predict future time increments. We propose to demonstrate the efficiency of the m-tensor regression to identify this dynamics. For small errors in the identified model, the approximation might diverge extremely fast from the real dynamics, therefore this represents a good way to demonstrate the robustness of our approach.

7.2.1. 3D Lorenz attractor. In a first example to describe the approach in a didactic way and demonstrate the robustness of our method, we build the regression of a 3D Lorenz attractor [20]. The models provided by the different regularization proposed before are then used to integrate the system in time on randomized initial conditions.

The model is 3D, *i.e.* $\mathbf{x} \in \mathbb{R}^3$ and f is defined as

$$\begin{cases} \frac{\partial [\mathbf{x}]_1}{\partial t} = \sigma([\mathbf{x}]_2 - [\mathbf{x}]_1) \\ \frac{\partial [\mathbf{x}]_2}{\partial t} = \rho[\mathbf{x}]_1 - [\mathbf{x}]_2 - [\mathbf{x}]_1[\mathbf{x}]_3 \\ \frac{\partial [\mathbf{x}]_3}{\partial t} = [\mathbf{x}]_1[\mathbf{x}]_2 - \beta[\mathbf{x}]_3 \end{cases}.$$

The parameters are chose an $\sigma = 10$, $\rho = 28$ and $\beta = 2.667$. The system is initialized as $\mathbf{x}(t=0) = [0 \quad 1 \quad 1.05]^\top$ and solved in time with the explicit scheme

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + f(\mathbf{x}(t))\Delta t.$$

We chose a timestep $\Delta t = 0.001$ and evaluate the behaviour of the system over $m = 500$ steps. To apply the m-tensor methodology, we chose to define the mapping $\boldsymbol{\psi}(x) = [1 \quad x]$ which allows us to build the m-tensor Φ as

$$\Phi = \bigotimes_{i=1}^3 \Psi_i$$

where we define row j of core i , denoted by $[\Psi_i]_j$, as

$$\forall i, 1 \leq i \leq 3, \forall j, 1 \leq j \leq m, [\Psi_i]_j = \boldsymbol{\psi}([\mathbf{x}_j]_i).$$

We then seek to predict the derivative $\frac{\partial \mathbf{x}}{\partial t}$ from the current state $\mathbf{x}$. For that reason, we build the right hand side of the system to solve as the matrix

$$\forall i, 1 \leq i \leq 3, \forall j, 1 \leq j \leq m, [Y]_{ji} = \frac{\partial [\mathbf{x}_j]_i}{\partial t}.$$

The resolution then follows the approaches presented before to give an approximation of f . This implicitly solves for

$$\Phi \times \mathbf{C} = Y$$

by building the approximation of f as

$$\hat{f}(\mathbf{x}) = \langle \varphi(\mathbf{x}) \times \Phi, Z \rangle$$

where we define the matrix Z based on the regularization we want to apply and $\varphi(\mathbf{x}) = \bigotimes_{i=1}^3 \psi([\mathbf{x}]_i)$. We present our results for the least squares, spectral truncation and almost linearly independent regressions.

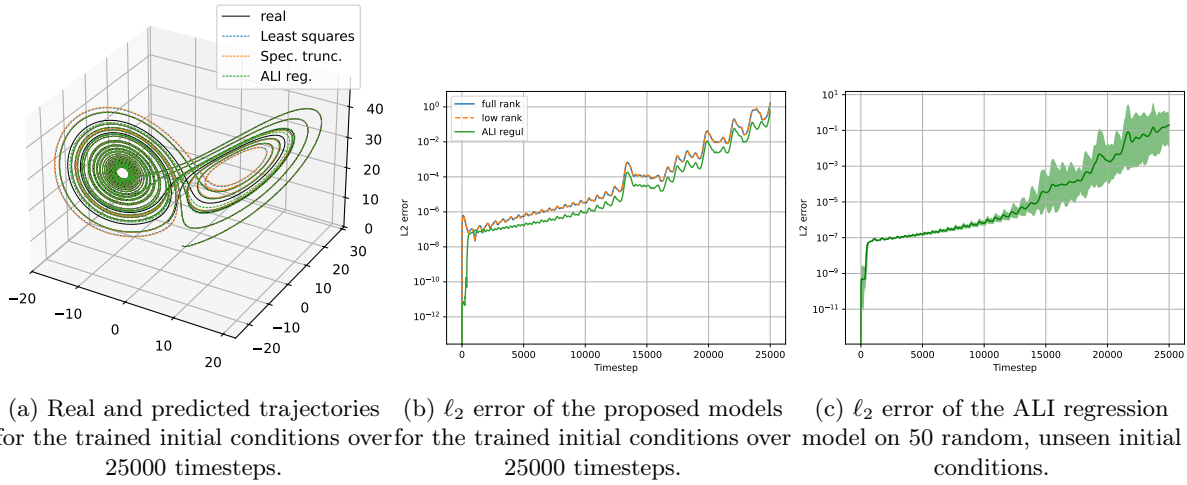


Figure 7.4: Evaluation of proposed regression models on a Lorenz attractor from a training on 500 timesteps.

We show the results of our method in figure 7.4. Figure 7.4a shows the trajectories followed by all three models and the real one on the trained initial conditions for 25000 timesteps, *i.e.* 50 times the trained period. Figure 7.4b presents the ℓ_2 error over the same span, still on the trained initial conditions. The error considered is defined by denoting $\mathbf{x}$ and $\hat{\mathbf{x}}$ the position and its approximation respectively

$$E_2 = \frac{\|\mathbf{x} - \hat{\mathbf{x}}\|_2}{\|\mathbf{x}\|_2}.$$

These results already demonstrate the robustness of the learnt model but we go further in generalizability by testing the ALI regression on 50 randomly selected initial conditions and plot the ℓ_2 error of the trajectories over a span of 25000 timesteps. This is the plot of figure 7.4c. The error is computed at each timestep with the same definition as before.

The only way to make these models more robust would be to eliminate fully the low coefficient that ultimately add up to make the model move away from the real trajectory. This translates into enforcing a ℓ_0 norm minimization which is an ongoing topic of research for this dimensionality of problems.

7.2.2. Kuramoto oscillators. The next example leverages the ability of the proposed tensor regression framework to scale well with dimension with only a few data points as input. We therefore take interest in the Kuramoto oscillator model [18]. The model is defined for an arbitrary number N of oscillators, all coupled through the equations

$$\frac{\partial [\mathbf{x}]_i}{\partial t} = \omega_i + \frac{K}{N} \sum_{j=1}^N \sin([\mathbf{x}]_j - [\mathbf{x}]_i).$$

The factor K is a coupling constant chosen here to be shared by all oscillators, each described by their phase $[\mathbf{x}]_{1 \leq i \leq N}$ (the state we estimate), and their natural frequency $\omega_{1 \leq i \leq N}$. We define the natural frequencies randomly in the interval $[-5, 5]$ and set the coupling factor $K = 2$.

We build our model by including bases ψ_1 and ψ_2 defined for each variable as

$$\psi_1(x) = [1 \quad \sin(x)] \quad \text{and} \quad \psi_2(x) = [1 \quad \cos(x)].$$

The resulting m-tensor is therefore built as a larger m-tensor product of two bases by variable. By denoting the cores built on each input for both basis $\Psi_{1 \leq i \leq N}^1$ and $\Psi_{1 \leq i \leq N}^2$, the m-tensor of the regression problem is built as

$$\Phi = \left(\bigotimes_{i=1}^N \Psi_i^1 \right) \otimes \left(\bigotimes_{i=1}^N \Psi_i^2 \right).$$

The ability of our method to produce robust models is tested by building for dimensions $N \in \{3, 10, 100\}$ a model, 5 times each to evaluate average results. Each model is built with different random natural frequencies and initial conditions. We plot the ℓ_2 errors for predicted phases in time over 50000 timesteps in figure 7.5 for $N = 3, 10, 100$ respectively in figures 7.5a, 7.5b and 7.5c. Results are based on the same training period of 1000 timesteps and then

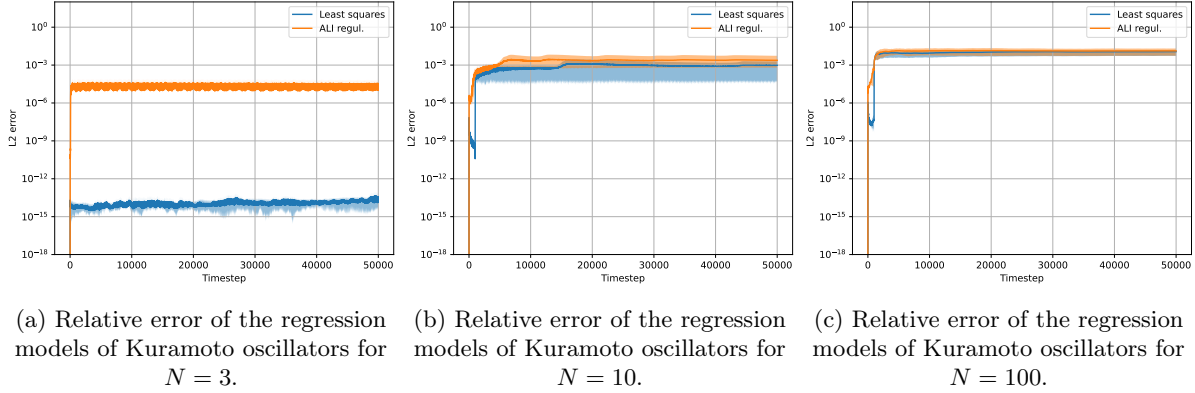


Figure 7.5: Relative errors of the least squares and ALI regression models for Kuramoto oscillators for $N \in \{3, 10, 100\}$ over 50000 timesteps from 1000 seen timesteps.

run using the same initial conditions as for training. Area between min and max ℓ_2 relative error are plotted around averaged results over the 5 run. We can see that ALI regression generally performs worse but systematically runs faster by retaining only 65, 360 and 370 (rounded averages) rows of Φ for $N = 3, 10, 100$ respectively. The growing error through dimensions is explained by the fact that the amount of given data does not evolve. Models trained over 1000 timesteps in the case $N = 100$ are massively underdetermined and still perform at about 2% error.

7.3. A practical note. A detail left until now in the implementation is that of numerical overflow. If we leverage the kernel trick to avoid computing all elements of the tensors at hand, it is strictly equivalent in practice. This means that working with a problem involving a dot product between two rank-1 tensors of order N and dimension $(p, p, \dots, p)$ will effectively lead to a result that multiplies and sums p^N elements together. This leads to results that are unmanageable numerically. Therefore the one-dimensional mappings ψ are built with a hyperparameter that scales the input down in high-dimensional contexts.

This hyperparameter is inherent with the method and a right tuning provides massive gains in term of precision. As it results purely from numerical issues, *i.e.* the mathematical background is sound otherwise, there must exist a normalization dependent on the input to be systematically applied, however this normalization has not yet been determined. In what follows, bases in low dimension (<10) are not scaled down, but higher dimensions are scaled with a factor 10^{-7} .

The real significant choice that remains is the choice of one-dimensional basis ψ . This choice of basis function can be made different for each axis depending on the problem at hand but the first choice we have systematically made was to start with

$$\psi(x) = [1 \quad x].$$

This choice, although simple builds all combinations of parameters at the lowest computational cost. If more precision is sought, the degree can be adapted incrementally, seeing how low the cost is to compute a solution. Cyclic behaviour can be represented using a sinusoidal polynomial basis or Fourier basis. A solution left unexplored in this paper is the choice of local bases along each axis. One may *discretize* each axis with local basis, even with compact support. The resulting tensor regression operator will capture localized effects in the N -dimensional mesh that is implicitly built.

All examples presented here are publicly available from data generation to result plotting. The implementation uses a toolbox leveraging the previously presented operations using m-tensors. Curious readers can access this code at: <https://github.com/RemiC-OV/MTensor>. The examples and tools presented were developed fully in a NumPy implementation, it is therefore simplistic and can be optimized. We leveraged algebraic structures to gain in memory use and computational efficiency but we did not implement any form of parallelism. It is one of the main development perspectives to demonstrate the gains possible. Most operations presented in this paper are parallelizable throughout dimensions and/or samples.

8. Conclusion and perspectives. In this paper we presented a tensor formalism for high dimensional non-linear regression. The introduced *m-tensor format* is tailored for regression in scarce data contexts. By leveraging tensor algebra, we provided properties from kernel methods with good scalability. The formalism smoothly generalizes well-known properties of matrix-based approaches to provide optimality to tensor structured models. We generalized different regularization techniques that seek to make models robust and generalizable from the low amount of data available. Those regularization techniques prove robust and yield low errors while limiting the scaling when faced with a high number of parameters. The proposed ALI regularization technique allows for further gains in computational and memory costs, achieving linear scaling at online stage on the provided examples.

The m-tensor formalism was shown to break the curse of dimensionality with, in the worst case, cubic scaling on the number of parameters. Linear scaling of memory use with the dimension of the problem is inherited from the separated variables approach as well as the parallel capability.

The proposed approach was applied on a general benchmark to demonstrate scaling and low relative errors. Robustness was demonstrated by applying the different regularization techniques to identify dynamical behaviour of two systems. In the first case, we show good generalizability for a Lorenz chaotic system. In the second we show again both generalizability and scalability up to 100 parameters with limited data.

In the future, we will seek different development, most probably stemming from randomized numerical linear algebra to further accelerate our tools and enhance their scaling. The same approach might lead us to ℓ_0 norm minimization. We will seek to expand the validity domain by exploiting rational form of approximations. On the application side, the tool seems most adapted to model order reduction, data-driven operator inference, large-scale optimization or classification. Generalization of the m-tensor formalism might prove useful also for solving parametric PDEs, at this stage most likely through a collocation method.

Acknowledgements. We acknowledge fundings from the ENSAM RTE Chair, insightful discussions with Antonio Falcò and rich and fruitful exchanges with Jad Mounayer.

Bibliography.

- [1] A. Ammar et al. “A new family of solvers for some classes of multidimensional partial differential equations encountered in kinetic theory modeling of complex fluids”. In: *Journal of Non-Newtonian Fluid Mechanics* 139.3 (2006), pp. 153–176. ISSN: 0377-0257. DOI: [10.1016/j.jnnfm.2006.07.007](https://doi.org/10.1016/j.jnnfm.2006.07.007).
- [2] Amine Ammar, Francisco Chinesta, and Antonio Falcò. “On the Convergence of a Greedy Rank-One Update Algorithm for a Class of Linear Systems”. In: *Arch Computat Methods Eng* 17 (2010), pp. 473–486. DOI: [10.1007/s11831-010-9048-z](https://doi.org/10.1007/s11831-010-9048-z).
- [3] Peter J. Baddoo et al. “Kernel learning for robust dynamic mode decomposition: linear and nonlinear disambiguation optimization”. In: *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 478.2260 (2022), p. 20210830. DOI: [10.1098/rspa.2021.0830](https://doi.org/10.1098/rspa.2021.0830).
- [4] R. Bellman, Rand Corporation, and Karreman Mathematics Research Collection. *Dynamic Programming*. Rand Corporation research study. Princeton University Press, 1957. ISBN: 9780691079516.
- [5] Gregory Beylkin and Martin J. Mohlenkamp. “Algorithms for Numerical Analysis in High Dimensions”. In: *SIAM Journal on Scientific Computing* 26.6 (2005), pp. 2133–2159. DOI: [10.1137/040604959](https://doi.org/10.1137/040604959).
- [6] Domenico Borzacchiello, José V. Aguado, and Francisco Chinesta. “Non-intrusive Sparse Subspace Learning for Parametrized Problems”. In: *Archives of Computational Methods in Engineering* 26 (2019), pp. 303–326. ISSN: 1886-1784. DOI: [10.1007/s11831-017-9241-4](https://doi.org/10.1007/s11831-017-9241-4).
- [7] Steven L. Brunton and J. Nathan Kutz. *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. 2nd ed. Cambridge University Press, 2022. DOI: [10.1017/9781009089517](https://doi.org/10.1017/9781009089517).

- [8] Y. Engel, S. Mannor, and R. Meir. “The kernel recursive least-squares algorithm”. In: *IEEE Transactions on Signal Processing* 52.8 (2004), pp. 2275–2285. DOI: [10.1109/TSP.2004.830985](https://doi.org/10.1109/TSP.2004.830985).
- [9] Patrick Gelß et al. “Feature space approximation for kernel-based supervised learning”. In: *Knowledge-Based Systems* 221 (2021), p. 106935. ISSN: 0950-7051. DOI: [10.1016/j.knosys.2021.106935](https://doi.org/10.1016/j.knosys.2021.106935).
- [10] Patrick Gelß et al. “Multidimensional Approximation of Nonlinear Dynamical Systems”. In: *Journal of Computational and Nonlinear Dynamics* 14.6 (Apr. 2019), p. 061006. ISSN: 1555-1415. DOI: [10.1115/1.4043148](https://doi.org/10.1115/1.4043148).
- [11] S.A. Goreinov, E.E. Tyrtyshnikov, and N.L. Zamarashkin. “A theory of pseudoskeleton approximations”. In: *Linear Algebra and its Applications* 261.1 (1997), pp. 1–21. ISSN: 0024-3795. DOI: [10.1016/S0024-3795\(96\)00301-1](https://doi.org/10.1016/S0024-3795(96)00301-1).
- [12] Tamara G. Kolda Grey Ballard. *Tensor Decompositions for Data Science*. Cambridge University Press, 2025. ISBN: 9781009471671.
- [13] Wolfgang Hackbusch. *Tensor Spaces and Numerical Tensor Calculus*. 2nd ed. Vol. 42. Springer Cham, 2019. ISBN: 978-3-030-35553-1. DOI: [10.1007/978-3-030-35554-8](https://doi.org/10.1007/978-3-030-35554-8).
- [14] Rubén Ibáñez et al. “A Multidimensional Data-Driven Sparse Identification Technique: The Sparse Proper Generalized Decomposition”. In: *Complexity* 2018 (2018), pp. 1–11. DOI: [10.1155/2018/5608286](https://doi.org/10.1155/2018/5608286).
- [15] Kenneth L. Judd et al. “Smolyak method for solving dynamic economic models: Lagrange interpolation, anisotropic grid and adaptive domain”. In: *Journal of Economic Dynamics and Control* 44.C (2014), pp. 92–123. DOI: [10.1016/j.jedc.2014.03.00](https://doi.org/10.1016/j.jedc.2014.03.00).
- [16] Stefan Klus and Patrick Gelß. “Tensor-Based Algorithms for Image Classification”. In: *Algorithms* 12.11 (2019). ISSN: 1999-4893. DOI: [10.3390/a12110240](https://doi.org/10.3390/a12110240).
- [17] Tamara G. Kolda and Brett W. Bader. “Tensor Decompositions and Applications”. In: *SIAM Review* 51.3 (2009), pp. 455–500. DOI: [10.1137/07070111X](https://doi.org/10.1137/07070111X).
- [18] Yoshiki Kuramoto. “Self-entrainment of a population of coupled non-linear oscillators”. In: *International Symposium on Mathematical Problems in Theoretical Physics*. Ed. by Huzihiro Araki. Berlin, Heidelberg: Springer Berlin Heidelberg, 1975, pp. 420–422. ISBN: 978-3-540-37509-8.
- [19] Charles F. Van Loan. “The ubiquitous Kronecker product”. In: *Journal of Computational and Applied Mathematics* 123.1 (2000). Numerical Analysis 2000. Vol. III: Linear Algebra, pp. 85–100. ISSN: 0377-0427. DOI: [10.1016/S0377-0427\(00\)00393-9](https://doi.org/10.1016/S0377-0427(00)00393-9).
- [20] Edward N. Lorenz. “Deterministic Nonperiodic Flow”. In: *Journal of Atmospheric Sciences* 20.2 (1963), pp. 130–141. DOI: [10.1175/1520-0469\(1963\)020<0130:DNF>2.0.CO;2](https://doi.org/10.1175/1520-0469(1963)020<0130:DNF>2.0.CO;2).
- [21] Michael W. Mahoney and Petros Drineas. “CUR matrix decompositions for improved data analysis”. In: *Proceedings of the National Academy of Sciences* 106.3 (2009), pp. 697–702. DOI: [10.1073/pnas.0803205106](https://doi.org/10.1073/pnas.0803205106).
- [22] James Mercer. “XVI. Functions of positive and negative type, and their connection the theory of integral equations”. In: *Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character* 209.441-458 (1909), pp. 415–446. DOI: [10.1098/rsta.1909.0016](https://doi.org/10.1098/rsta.1909.0016).
- [23] I. V. Oseledets. “Tensor-Train Decomposition”. In: *SIAM Journal on Scientific Computing* 33.5 (2011), pp. 2295–2317. DOI: [10.1137/090752286](https://doi.org/10.1137/090752286).
- [24] R. Penrose. “A generalized inverse for matrices”. In: *Mathematical Proceedings of the Cambridge Philosophical Society* 51.3 (1955), pp. 406–413. DOI: [10.1017/S0305004100030401](https://doi.org/10.1017/S0305004100030401).
- [25] Bernhard Schölkopf, Alexander Smola, and Klaus-Robert Müller. “Nonlinear Component Analysis as a Kernel Eigenvalue Problem”. In: *Neural Computation* 10.5 (July 1998), pp. 1299–1319. ISSN: 0899-7667. DOI: [10.1162/089976698300017467](https://doi.org/10.1162/089976698300017467).
- [26] Bernhard Schölkopf and Alexander J. Smola. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. The MIT Press, Dec. 2001. ISBN: 9780262256933. DOI: [10.7551/mitpress/4175.001.0001](https://doi.org/10.7551/mitpress/4175.001.0001).
- [27] V.I. Slyusar. “A family of face products of matrices and its properties.” In: *Cybern Syst Anal* (1999), pp. 379–384. DOI: [10.1007/BF02733426](https://doi.org/10.1007/BF02733426).

Appendix A. Tensor operations complexity analysis. In this section we demonstrate the low complexity of the tensor operations previously introduced. Those are what makes the proposed tensor formalism efficient and constitute the main tricks we leverage. In practice those mostly rely on the low computational complexity of rank-1 tensors.

In what follows, we consider that for a given matrix A or m-tensor $\mathbf{T}$, $[A]_k$ and $[\mathbf{T}]_k$ respectively denotes the k -th row (vector) of A and k -th row (rank-1 tensor) of $\mathbf{T}$. We defined in section 3 two specific tensor contractions, in this section we define a more general operator. Let $\mathbf{T} \in \mathbb{R}^{p_1 \times p_2 \times \dots \times p_N}$ denote an arbitrary order N tensor. Then the contraction operations are defined as summations over one or more dimensions of the tensor. We denote $\text{contract}_{k_1, k_2, \dots, k_n}(\cdot)$ the operator computing the contraction of a tensor over axes $k_1, k_2, \dots, k_n$. Elements of the resulting tensor are defined as

$$\begin{aligned} [\text{contract}_{k_1, k_2, \dots, k_n}(\mathbf{T})]_{i_1 i_2 \dots i_{k_1-1} i_{k_1+1} \dots i_{k_2-1} i_{k_2+1} \dots i_{k_n-1} i_{k_n+1} \dots i_N} = \\ \sum_{j_1=1}^{p_{k_1}} \sum_{j_2=1}^{p_{k_2}} \dots \sum_{j_n=1}^{p_{k_n}} [\mathbf{T}]_{i_1 i_2 \dots i_{k_1-1} j_1 i_{k_1+1} \dots i_{k_2-1} j_2 i_{k_2+1} \dots i_{k_n-1} j_n i_{k_n+1} \dots i_N} \\ \forall i_\kappa, 1 \leq i_\kappa \leq p_\kappa, \forall \kappa, 1 \leq \kappa \leq N, \kappa \notin \{k_1, k_2, \dots, k_n\}. \end{aligned}$$

And $\text{contract}_{k_1, k_2, \dots, k_n}(\mathbf{T}) \in \mathbb{R}^{p_1 \times p_2 \times \dots \times p_{k_1-1} \times 1 \times p_{k_1+1} \times \dots \times p_{k_2-1} \times 1 \times p_{k_2+1} \times \dots \times p_{k_n-1} \times 1 \times p_{k_n+1} \times \dots \times p_N}$, where the contracted dimension are reduced to 1.

A.1. Rank-1 tensor contraction. We highlight the fact that the contraction of a rank-1 tensor consists in the product of the contractions over each axes on their own. Let $\mathbf{T} = \bigotimes_{i=1}^N \mathbf{u}_i$ with $\mathbf{u}_i \in \mathbb{R}^{p_i}$, then its contraction is computed as

$$\begin{aligned} \text{contract}_{1,2,\dots,N}(\mathbf{T}) &= \text{contract}_{1,2,\dots,N} \left(\bigotimes_{i=1}^N \mathbf{u}_i \right), \\ &= \sum_{j_1=1}^{p_1} \sum_{j_2=1}^{p_2} \dots \sum_{j_N=1}^{p_N} \left[\bigotimes_{i=1}^N \mathbf{u}_i \right]_{j_1 j_2 \dots j_N}, \\ &= \sum_{j_1=1}^{p_1} \sum_{j_2=1}^{p_2} \dots \sum_{j_N=1}^{p_N} \prod_{i=1}^N [\mathbf{u}_i]_{j_i}, \\ &= \sum_{j_1=1}^{p_1} [\mathbf{u}_1]_{j_1} \sum_{j_2=1}^{p_2} [\mathbf{u}_2]_{j_2} \dots \sum_{j_N=1}^{p_N} [\mathbf{u}_N]_{j_N}, \\ &= \prod_{i=1}^N \text{contract}_1(\mathbf{u}_i). \end{aligned}$$

If we consider $p_i = p_j = p$ for ease of notation, this results in a complexity of $\mathcal{O}(Np)$.

A.2. Rank-1 tensors Hadamard product. We highlight a property of the Hadamard product of rank-1 tensors. Let $\forall i, 1 \leq i \leq M$, $\mathbf{T}_i = \bigotimes_{j=1}^N \mathbf{u}_j^i$ with $\mathbf{u}_j^i \in \mathbb{R}^{p_j}$ for any arbitrary number M of rank-1 tensors. We demonstrate that the Hadamard product of M rank-1 tensors is the tensor product of Hadamard products:

$$\begin{aligned} \bigodot_{i=1}^M \mathbf{T}_i &= \bigodot_{i=1}^M \bigotimes_{j=1}^N \mathbf{u}_j^i, \\ &= \bigotimes_{j=1}^N \bigodot_{i=1}^M \mathbf{u}_j^i. \end{aligned}$$

We can prove it by analyzing the behaviour of each component of the Hadamard product:

$$\left[\bigodot_{i=1}^M \mathbf{T}_i \right]_{k_1 k_2 \dots k_N} = \prod_{i=1}^M \prod_{j=1}^N [\mathbf{u}_j^i]_{k_j}.$$

The product here being commutative, we can interchange Hadamard and tensor products.

A.3. M-product complexity. In this section we demonstrate the low complexity of the m-product inherited by the properties of the rank-1 tensor contraction and the commutativity of Hadamard and tensor products for rank-1 tensors. Let us consider the two m-tensors $\mathbf{T}_1 \in \mathbb{R}^{m_1 \times p_1 \times p_2 \times \dots \times p_{N_1}}$ and $\mathbf{T}_2 \in \mathbb{R}^{m_2 \times q_1 \times q_2 \times \dots \times q_{N_2}}$ be m-tensors respectively based on cores $\{A_i^1\}_{1 \leq i \leq N_1}$ and $\{A_i^2\}_{1 \leq i \leq N_2}$.

$$\mathbf{T}_1 = \bigotimes_{i=1}^{N_1} A_i^1, \quad \mathbf{T}_2 = \bigotimes_{i=1}^{N_2} A_i^2.$$

Provided that both tensors have the same order (*i.e.* $N_1 = N_2 = N$) and same row dimensions *i.e.* $\forall i, 1 \leq i \leq N, p_i = q_i$. We can build an element of $\mathbf{T}^1 \ltimes (\mathbf{T}^2)^\top \in \mathbb{R}^{m_1 \times m_2}$ as the inner product of order $N - 1$ rank-1 tensors. Elements of that product are defined as

$$\begin{aligned} [\mathbf{T}_1 \ltimes \mathbf{T}_2^\top]_{k_1 k_2} &= \langle [\mathbf{T}_1]_{k_1}, [\mathbf{T}_2]_{k_2} \rangle, \\ &= \sum_{i_1=1}^{p_1} \sum_{i_2=1}^{p_2} \dots \sum_{i_N=1}^{p_N} [\mathbf{T}_1]_{k_1 i_1 i_2 \dots i_N} [\mathbf{T}_2]_{k_2 i_1 i_2 \dots i_N}, \\ &= \text{contract}_r([\mathbf{T}_1]_{k_1} \odot [\mathbf{T}_2]_{k_2}), \\ &= \text{contract}_r \left(\left(\bigotimes_{i=1}^N [A_i^1]_{k_1} \right) \odot \left(\bigotimes_{i=1}^N [A_i^2]_{k_2} \right) \right), \\ &= \text{contract}_r \left(\bigotimes_{i=1}^N [A_i^1]_{k_1} \odot [A_i^2]_{k_2} \right), \\ &= \prod_{i=1}^N \text{contract}([A_i^1]_{k_1} \odot [A_i^2]_{k_2}), \\ &= \prod_{i=1}^N [A_i^1]_{k_1} \cdot [A_i^2]_{k_2}. \end{aligned}$$

The overall computation results in the Hadamard product of matrix product of corresponding cores of both m-tensors:

$$\mathbf{T}_1 \ltimes \mathbf{T}_2^\top = \bigodot_{i=1}^N A_i^1 (A_i^2)^\top.$$

Considering, for ease of notation that $m_1 = m_2 = m$ and $\forall i, 1 \leq i \leq N, p_i = p$ the complexity of the m-product is of $\mathcal{O}(m^2 p N)$. This complexity scales only linearly with dimension N and is dominated by m^2 . The N-way contracted product of arbitrary tensors with the same dimensions would result in a complexity of $\mathcal{O}(m^2 p^N)$.

Appendix B. Algorithms to compute ALI decomposition. In this section we provide algorithms to compute the ALI decomposition of m-tensors. As we stated in the document, we have two ways of constructing it. The first is the greedy way, inspired from online learning of streamed data. We qualify the second as the optimal decomposition. We first review the greedy way and then provide arguments for the more costly optimal decomposition. We note that review of randomized numerical linear algebra and the work of Mahoney and Drineas on CUR matrix decomposition [21] lead us to seek a *near optimal* algorithm leveraging randomized approaches with a low computational cost.

As a reminder and to settle notations for this section, we consider an m-tensor $\mathbf{T}$ with c-dim m and provide its ALI decomposition with a tolerance ε as

$$\mathbf{T} \approx W \tilde{\mathbf{T}}, \quad \|\mathbf{T} - W \tilde{\mathbf{T}}\|^2 \leq \varepsilon.$$

Consider we are given a tensor mapping φ . From high-dimensional data in a set of points $\mathcal{P}$ from a manifold $\mathcal{M} \subset \mathbb{R}^N$, we seek a minimal subspace $\tilde{\mathcal{P}}$ of size $\tilde{m}$. Then we can capture properties of $\mathcal{M}$ or provide a model for it with low dimensionality.

In the remainder of the section, we give a geometric interpretation to the incremental construction of the subset of rows from a given m-tensor. We consider each row to be a point (vector) in a high dimensional space. Providing the distance definition from the inner product of two of those points, *i.e.* the inner product of rank-1 tensors, we seek to maximize at each step the distance to the previous subspace.

B.1. Greedy decomposition. The greedy construction of the ALI decomposition of an m-tensor considers that we sweep once through the rows of an m-tensor $\mathbf{T}$ or treat rows as they are streamed (or equivalently the rank-1 tensor mapped from streamed data) to select the best ones on-the-fly. This is considered a greedy approach because it builds the best subset from available data as it comes.

Element 1. The first provided row $[\mathbf{T}]_1$ is used by default to initialize $\tilde{\mathbf{T}}$. Weights W_1 are initialized as an array containing only 1. By convenience we define

$$L_1 = \|[\mathbf{T}]_1\|.$$

From that given point, we build a subspace $\text{span}(\{[\mathbf{T}]_1\})$. Now for the next point, we evaluate the distance to that subspace. For a new row $[\mathbf{T}]_2$, we compute δ defined as

$$\delta = \min_{w \in \mathbb{R}} \|[\mathbf{T}]_2 - w[\mathbf{T}]_1\|^2.$$

Given by the minimizing w defined as

$$w = \frac{\langle [\mathbf{T}]_1, [\mathbf{T}]_2 \rangle}{\|[\mathbf{T}]_1\|^2}.$$

This minimizer is obtained by expanding the norm in the definition of δ and deriving the minimum of a quadratic polynomial in w . Using that minimizer, δ will be given by

$$\delta = \|[\mathbf{T}]_2\|^2 - \left(\frac{\langle [\mathbf{T}]_2, [\mathbf{T}]_1 \rangle}{\|[\mathbf{T}]_1\|^2} \right)^2.$$

By comparing δ to the precision criterion ε , the point is kept or not. If it is kept, $[\mathbf{T}]_2$ is appended to $\tilde{\mathbf{T}}$ and L is updated as

$$L_2 = \begin{bmatrix} L_1 & 0 \\ s & \sqrt{\|[\mathbf{T}]_2\|^2 - s^2} \end{bmatrix},$$

where s is defined as

$$s = \frac{\langle [\mathbf{T}]_1, [\mathbf{T}]_2 \rangle}{L_1}.$$

The weight matrix W is updated as

$$W_2 = \begin{bmatrix} W_1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Otherwise, if $\delta < \varepsilon$, only the weight matrix is updated as

$$W_2 = \begin{bmatrix} W_1 \\ w \end{bmatrix}.$$

Element k . For any new row $[\mathbf{T}]_k$, we adopt the incremental construction for $\tilde{\mathbf{T}}$ as follows. Consider we retained $\tilde{k}$ elements in $\tilde{\mathbf{T}}$ until that point. This means we need to evaluate the distance of $[\mathbf{T}]_k$ to a $\tilde{k}$ -dimensional subspace. We define the vector $\mathbf{b}$ defined from the already selected rows and the new one

$$\mathbf{b} = \tilde{\mathbf{T}}[\mathbf{T}]_k^T.$$

The distance δ between $[\mathbf{T}]_k$ and $\text{span}(\tilde{\mathbf{T}})$ is evaluated as

$$\delta = \min_{\mathbf{w}} \left\| [\mathbf{T}]_k - \mathbf{w}^T \tilde{\mathbf{T}} \right\|^2 = \|[\mathbf{T}]_k\|^2 - \mathbf{b}^T \tilde{P}^{-1} \mathbf{b},$$

where $\tilde{P} = \tilde{\mathbf{T}}\tilde{\mathbf{T}}^T$ is symmetric positive definite and whose Cholesky factors are given by $L_{\tilde{k}}$, *i.e.*

$$\tilde{P} = L_{\tilde{k}} L_{\tilde{k}}^T.$$

Therefore we define the vector $\mathbf{s}$ from $L_{\tilde{k}}$ directly as

$$\mathbf{s} = L_{\tilde{k}}^{-1} \mathbf{b},$$

which can be computed by downward backsubstitution. Given the vector $\mathbf{s}$, the second term of the definition of δ can be efficiently computed as $\mathbf{b}^\top \tilde{P}^{-1} \mathbf{b} = \mathbf{s}^\top \mathbf{s}$. Then by confronting δ to the tolerance ε , $[\mathbf{T}]_k$ is kept or not. If $\delta > \varepsilon$, then $[\mathbf{T}]_k$ is appended to $\tilde{\mathbf{T}}$ and the Cholesky factor is updated as

$$L_{\tilde{k}+1} = \begin{bmatrix} L_{\tilde{k}} & 0 \\ \mathbf{s}^\top & \sqrt{\|[\mathbf{T}]_k\|^2 - \mathbf{s}^\top \mathbf{s}} \end{bmatrix}.$$

The weight matrix W is updated as

$$W_{k+1} = \begin{bmatrix} W_k & 0 \\ 0 & 1 \end{bmatrix}.$$

Otherwise, if $\delta < \varepsilon$, only the weight matrix is updated as

$$W_{k+1} = \begin{bmatrix} W_k \\ \mathbf{w}^\top \end{bmatrix},$$

with $\mathbf{w} = (L_k^\top)^{-1} \mathbf{s}$, which can be computed by upward backsubstitution.

This construction is excessively efficient as it loops only once on the rows of $\mathbf{T}$, it does not suffer from numerical instability as we avoid computing the inverse $\tilde{P}^{-1}$. Instead we leverage its incrementally constructed Cholesky factors $L_{\tilde{k}}$. For that reason it is fit for streamed data if the hyperparameter ε can be chosen a priori. We describe this construction in the pseudocode B.1.

Algorithm B.1 Greedy ALI decomposition

Input: $\varepsilon, [\mathbf{T}]_1$

Output: $\mathbf{T}, W$

Initialization

$\tilde{\mathbf{T}} \leftarrow [\mathbf{T}]_1$

▷ Initialize reduced m-tensor

$L_1 \leftarrow \|[\mathbf{T}]_1\|$

▷ Initialize Cholesky factor

$\tilde{k} \leftarrow 1$

▷ Initialize row index in reduced m-tensor

$W \leftarrow [1]$

▷ Initialize weights

for any new $[\mathbf{T}]_k, k > 1$ **do**

▷ Loop on rows of initial $\mathbf{T}$ or streamed data

 Compute $\mathbf{b}$

$\mathbf{b} \leftarrow \tilde{\mathbf{T}}[\mathbf{T}]_k^\top$

 Compute $\mathbf{s}$ by backsubstitution

$\mathbf{s} \leftarrow L_{\tilde{k}}^{-1} \mathbf{b}$

 Evaluate δ

$\delta \leftarrow \|[\mathbf{T}]_k\|^2 - \mathbf{s}^\top \mathbf{s}$

if $\delta > \varepsilon$ **then**

▷ Compare distance to threshold

 Update reduced m-tensor $\tilde{\mathbf{T}}$

$\tilde{\mathbf{T}} \leftarrow \begin{bmatrix} \tilde{\mathbf{T}} \\ [\mathbf{T}]_k \end{bmatrix}$

 Update number of kept tensors $\tilde{k}$

$\tilde{k} \leftarrow \tilde{k} + 1$

 Update Cholesky factor L

$L_{\tilde{k}} \leftarrow \begin{bmatrix} L_{\tilde{k}-1} & 0 \\ \mathbf{s}^\top & \sqrt{\|[\mathbf{T}]_k\|^2 - \mathbf{s}^\top \mathbf{s}} \end{bmatrix}$

 Update weights W

$W \leftarrow \begin{bmatrix} W & 0 \\ 0 & 1 \end{bmatrix}$

else

 Update weights W

$W \leftarrow \begin{bmatrix} W \\ (L_k^\top)^{-1} \mathbf{s} \end{bmatrix}$

end if

end for

return $W, \tilde{\mathbf{T}}$

In their work, Engel et al. [8] give a proof that the number of kept tensors $\tilde{k}$ will always be finite under mild conditions. This means that the algorithm will eventually stop adding tensors to $\tilde{\mathbf{T}}$.

B.2. Optimal decomposition. Consider we are given a full m-tensor $\mathbf{T}$ with c-dim m . We seek the m-tensor $\tilde{\mathbf{T}}$ of c-dim $\tilde{m} < m$ such that all rows of $\mathbf{T}$ are *almost linearly dependent* on the rows of $\tilde{\mathbf{T}}$ in the ℓ_2 error sense with respect to a tolerance ε . To build such optimal decomposition, we propose an iterative approach during which we add the best choice among all rows of $\mathbf{T}$ at each step.

Element 0. The first row of $\tilde{\mathbf{T}}$ is selected so that its span minimizes the sum of distances with the other rows. To evaluate the distance of all points to the subspaces spanned by each points, we build the matrix $\Delta_0 \in \mathbb{R}^{m \times m}$ as

$$[\Delta_0]_{ij} = \delta_{ij}^0, \quad \delta_{ij}^0 = \min_{w \in \mathbb{R}} \|\mathbf{T}_i - w[\mathbf{T}]_j\|^2.$$

Each element is defined by expanding the norm in the definition of distances provided here. This expansion provides a quadratic formulation in terms of w which admits a unique minimum that we inject to build Δ_0 with elements defined as

$$\delta_{ij}^0 = \|\mathbf{T}_i\|^2 - \left(\frac{\langle [\mathbf{T}]_i, [\mathbf{T}]_j \rangle}{\|[\mathbf{T}]_j\|} \right)^2.$$

We compute the contraction of each column vector of Δ_0 and select the first row of $\tilde{\mathbf{T}}$ to minimize that sum. If all distances in the column vector corresponding to the selected point were below the precision tolerance ε , then $\tilde{\mathbf{T}}$ would be sufficient and the algorithm would end there.

In practice, the elements of the matrix Δ_0 can be built from the elements of $P = \mathbf{T}\mathbf{T}^\top$. The squared norms are on its diagonal and the inner products in the upper (or lower equivalently) diagonal. It is important to notice that Δ_0 is not symmetric.

Element k . The k -th row of $\tilde{\mathbf{T}}$, $k \geq 1$ is selected so that the subspace

$$\text{span} \left(\left\{ [\tilde{\mathbf{T}}]_i, 0 \leq i \leq k \right\} \right)$$

minimizes the distances to all other points. In order to evaluate these distances, we denote $\tilde{\mathbf{T}}_k$ the m-tensor $\mathbf{T}$ with rows previously selected in $\tilde{\mathbf{T}}$ removed. We define $\tilde{\mathbf{T}}_j$ as $\tilde{\mathbf{T}}$ appended with the j -th row of $\tilde{\mathbf{T}}_k$

$$\tilde{\mathbf{T}}_j = \begin{bmatrix} \tilde{\mathbf{T}} \\ [\mathbf{T}_k]_j \end{bmatrix}.$$

Then we build the matrix $\Delta_k \in \mathbb{R}^{(m-k) \times (m-k)}$ to evaluate the distances of each remaining point (not in $\tilde{\mathbf{T}}$) to each possible augmented subspaces.

$$[\Delta_k]_{ij} = \delta_{ij}^k, \quad \delta_{ij}^k = \min_{\mathbf{w} \in \mathbb{R}^k} \left\| [\tilde{\mathbf{T}}_k]_i - \mathbf{w}^\top \tilde{\mathbf{T}}_j \right\|^2.$$

The minimizing weights $\mathbf{w}$ can be computed by expanding the norm and solving for the unique minimum. This results in the minimized distances given by

$$\delta_{ij}^k = \|[\tilde{\mathbf{T}}_k]_i\|^2 - (\mathbf{b}_k^{ij})^\top (\tilde{P}_k^j)^{-1} \mathbf{b}_k^{ij},$$

where we define $\mathbf{b}_k^{ij} = \tilde{\mathbf{T}}_j [\tilde{\mathbf{T}}_k]_i^\top$, $\tilde{P}_k^j = \tilde{\mathbf{T}}_j \tilde{\mathbf{T}}_j^\top$. We notice that $\tilde{P}_k^j$ is symmetric positive definite, and therefore admits a Cholesky factorization $\tilde{P}_k^j = L_k^j (L_k^j)^\top$. Seeing that each distance computation requires the inversion of $\tilde{P}_k^j$, we leverage that factor's definition and reformulate the second term of the definition of δ_{ij}^k as

$$(\mathbf{b}_k^{ij})^\top (\tilde{P}_k^j)^{-1} \mathbf{b}_k^{ij} = ((L_k^j)^{-1} \mathbf{b}_k^{ij})^\top ((L_k^j)^{-1} \mathbf{b}_k^{ij}) = (\mathbf{s}_k^{ij})^\top \mathbf{s}_k^{ij}.$$

With this construction, a single upward backsubstitution is needed to compute each $\mathbf{s}_k^{ij}$. We propose to alleviate the notations and leverage those formulations. Therefore we suggest the following matrix construction for Δ_k for more readability. We define at step k the matrix as

$$\Delta_k = \text{diag}(\mathbf{n}) \mathbf{1}_{m-k} - S_k^\top S_k,$$

where $\mathbf{1}_{m-k}$ denotes the square matrix of size $m-k$ filled with ones. Then $\mathbf{n}$ is defined as the vector containing the norms of each row of $\tilde{\mathbf{T}}$, *i.e.*

$$\mathbf{n}^\top = [\|[\tilde{\mathbf{T}}]_1\| \quad \|[\tilde{\mathbf{T}}]_2\| \quad \dots \quad \|[\tilde{\mathbf{T}}]_{m-k}\|].$$

The matrix S_k is defined as

$$S_k = L_k^{-1} B_k,$$

where B_k is defined as

$$B_k = \tilde{\mathbf{T}} \tilde{\mathbf{T}}^\top,$$

and L_k is the Cholesky factor of $\tilde{P}_k = \tilde{\mathbf{T}} \tilde{\mathbf{T}}^\top$. As for the first element selection, we sum the columns of Δ_k to obtain a vector $\boldsymbol{\delta}_k$. We select the index of the best choice in the rows of $\tilde{\mathbf{T}}_k$ as the minimizer

$$\hat{i} = \operatorname{argmin}_i ([\boldsymbol{\delta}_k]_i).$$

The corresponding row of $\tilde{\mathbf{T}}_k$ is appended to $\tilde{\mathbf{T}}$. We check if all values of distances in the $\hat{i}$ -th column of Δ_k are below ε , in which case the precision criterion is reached and the algorithm is stopped. Otherwise, operators are updated and the loop continues.

We note that L_k can be built incrementally, along with $\tilde{\mathbf{T}}$ with $L_0 = [||[\tilde{\mathbf{T}}]_0||]$ and for $k \geq 1$ with the formula

$$L_{k+1} = \begin{bmatrix} L_k & 0 \\ \mathbf{s}_k^\top & \sqrt{||[\tilde{\mathbf{T}}]_k||^2 - \mathbf{s}_k^\top \mathbf{s}_k} \end{bmatrix},$$

where $\mathbf{s}_k$ is the $\hat{i}$ -th column vector of S_k .

Once the optimal $\tilde{\mathbf{T}}$ of size $\tilde{m}$ is computed, we evaluate the weight matrix using the projector onto the span of $\tilde{\mathbf{T}}$ as $\Pi_{\tilde{\mathbf{T}}}$ given by

$$\Pi_{\tilde{\mathbf{T}}} = \tilde{\mathbf{T}}^\top (\tilde{\mathbf{T}} \tilde{\mathbf{T}}^\top)^{-1} \tilde{\mathbf{T}}.$$

That projector is used to circumvent the fact that our decomposition does not provide an orthonormal basis. Therefore we cannot simply build the weights in a unique way without defining a way to project elements on the almost linearly independent basis. The final weights matrix W is built as

$$W = \mathbf{T} \tilde{\mathbf{T}}^\top \tilde{P}_m^{-1} = \mathbf{T} \tilde{\mathbf{T}}^\top (L_m^{-1})^\top L_m^{-1}.$$

We provide the pseudoalgorithm B.2 to build the optimal m-tensor ALI decomposition from a full m-tensor. The algorithm uses the Cholesky factor and updates it in the loop. Therefore it is already an optimized and numerically stable option to implement.

Remark B.1. We make a remark on the use in the context of regression. In that case, we consider we are given a full set of sample $\mathcal{P}$ of size m . We seek the subset $\tilde{\mathcal{P}}$ of size $\tilde{m}$ such that the span of the elements of $\tilde{\mathcal{P}}$ through the mapping φ minimizes the ℓ_2 error with the full mapped set with respect to a tolerance ε .

The first element $\mathbf{x}_0$ of $\tilde{\mathcal{P}}$ is selected so that the span of its mapping $\operatorname{span}(\{\varphi(\mathbf{x}_0)\})$ minimizes the sum of distances with the remainder of the mapped set. Then, the k -th element $\mathbf{x}_k$, $k \geq 1$ of the subset $\tilde{\mathcal{P}}$ is selected so that the subspace

$$\operatorname{span}\left(\left\{\varphi(\mathbf{x}), \mathbf{x} \in \tilde{\mathcal{P}} \cup \{\mathbf{x}_k\}\right\}\right)$$

minimizes the distances to all other mapped points. The approach is exactly the same and leverages the fact that the mapping φ maps a point to a rank-1 tensor and a set to an m-tensor. That way a row $[\mathbf{T}]_i$ of $\mathbf{T}$ is seen as the mapping of a point $\mathbf{x}_i \in \mathcal{P}$. The rows of $\tilde{\mathbf{T}}$ are the points of $\tilde{\mathcal{P}}$ mapped through φ .

Remark B.2. As stated earlier, we realized the gained to be found from the field of randomized numerical linear algebra. By seeking a m-tensor CUR generalization, we found in the work of Mahoney and Drineas [21] a path for perspectives leveraging randomized approaches, instead of what we have done that relates more with the pseudoskeleton approach of Goreinov et al. [11].

Algorithm B.2 Optimal ALI decomposition

Input: $\mathbf{T}, \varepsilon$ **Output:** $\tilde{\mathbf{T}}, W$ Build matrix Δ_0 Sum columns of Δ_0 $[\delta_0]_i \leftarrow \sum_{j=1}^m [\Delta_0]_{ji}$ Select first element of $\tilde{\mathbf{T}}$ $\tilde{\mathbf{T}} \leftarrow [\mathbf{T}]_{\hat{i}}, \hat{i} = \operatorname{argmin}_i([\delta_0]_i)$ Initialize Cholesky factor L_1 $L_1 \leftarrow [||\tilde{\mathbf{T}}||]$ Define reduced m-tensor $\bar{\mathbf{T}}_1$ by removing $\hat{i}$ -th row of $\mathbf{T}$ **for** $k, 1 \leq k \leq m$ **do** $\triangleright$ Loop on rows of $\mathbf{T}$ Build vector $\bar{\mathbf{n}}_k$ of norms of each row of $\bar{\mathbf{T}}_k$ $\bar{\mathbf{n}}_k^\top \leftarrow [||[\bar{\mathbf{T}}_k]_1|| \quad ||[\bar{\mathbf{T}}_k]_2|| \quad \dots \quad ||[\bar{\mathbf{T}}_k]_{m-k}||]$ Build matrix B_k $B_k \leftarrow \tilde{\mathbf{T}} \bar{\mathbf{T}}_k^\top$ Compute matrix S_k $S_k \leftarrow L_k^{-1} B_k$ Compute matrix Δ_k $\Delta_k \leftarrow \operatorname{diag}(\bar{\mathbf{n}}_k) \mathbf{1} - S_k^\top S_k$ Sum columns of Δ_k in vector δ_k $[\delta_k]_i \leftarrow \sum_{j=1}^{m-k} [\Delta_k]_{ji}$ Find optimal column index $\hat{i}$ $\hat{i} = \operatorname{argmin}_i([\delta_k]_i)$ Append $\hat{i}$ -th row of $\bar{\mathbf{T}}$ as k -th row of $\tilde{\mathbf{T}}$ $\tilde{\mathbf{T}}^\top \leftarrow [\tilde{\mathbf{T}}^\top, [\mathbf{T}]_{\hat{i}}^\top]$ Select $\hat{i}$ -th column vector of S_k $\mathbf{s}_k \leftarrow [S_k^\top]_{\hat{i}}$ Update Cholesky factor L_{k+1}
$$L_{k+1} \leftarrow \begin{bmatrix} L_k & 0 \\ \mathbf{s}_k^\top & \sqrt{||[\mathbf{T}]_{\hat{i}}||^2 - \mathbf{s}_k^\top \mathbf{s}_k} \end{bmatrix}$$
Update $\bar{\mathbf{T}}_{k+1}$ by removing $\hat{i}$ -th row of $\bar{\mathbf{T}}_k$ **if** $\forall j, 1 \leq j \leq m - k, [\Delta]_{j\hat{i}} \leq \varepsilon$ **then** $\triangleright$ If all distances to current subspace fall under ε

Break loop

end if**end for**Compute weights W with respect to final $\tilde{\mathbf{T}}$ $W \leftarrow \mathbf{T} \tilde{\mathbf{T}}^\top L_k^{-\top} L_k^{-1}$ **return** $\tilde{\mathbf{T}}, W$
